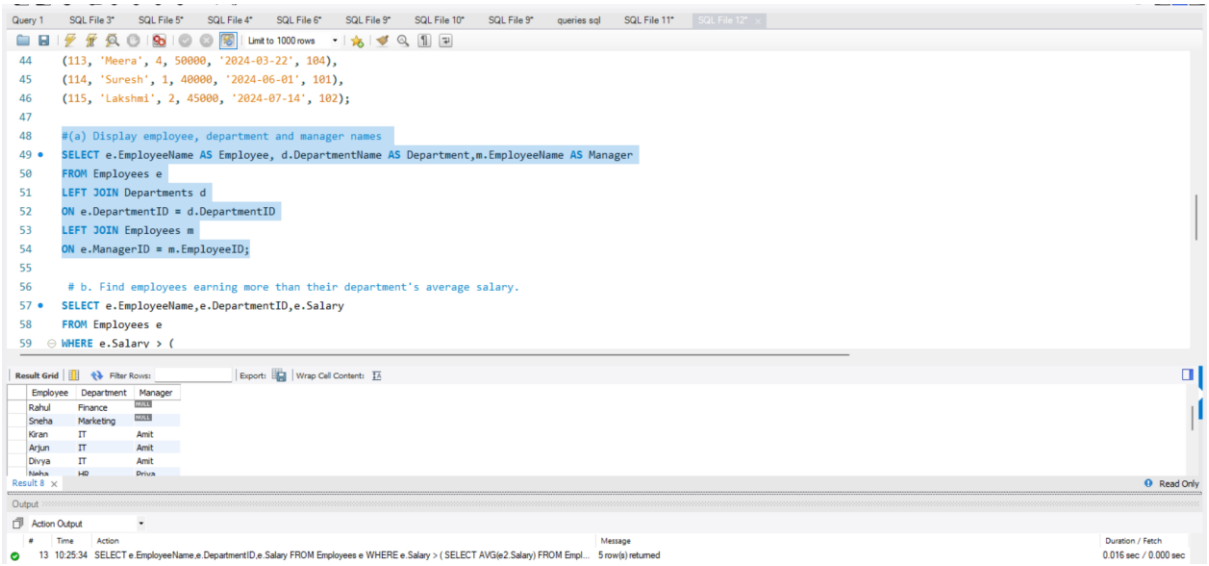
CLOUD DATA ENGINEER ASSESSMENT

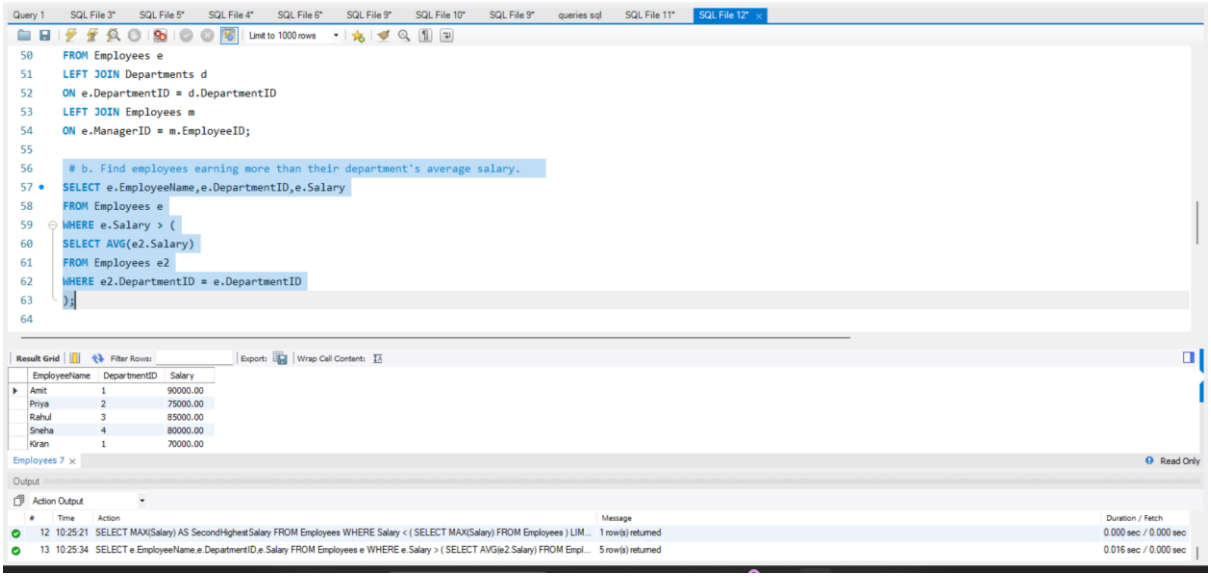
SECTION A: ADVANCED SQL

Question 1: Database Design and Advanced Queries

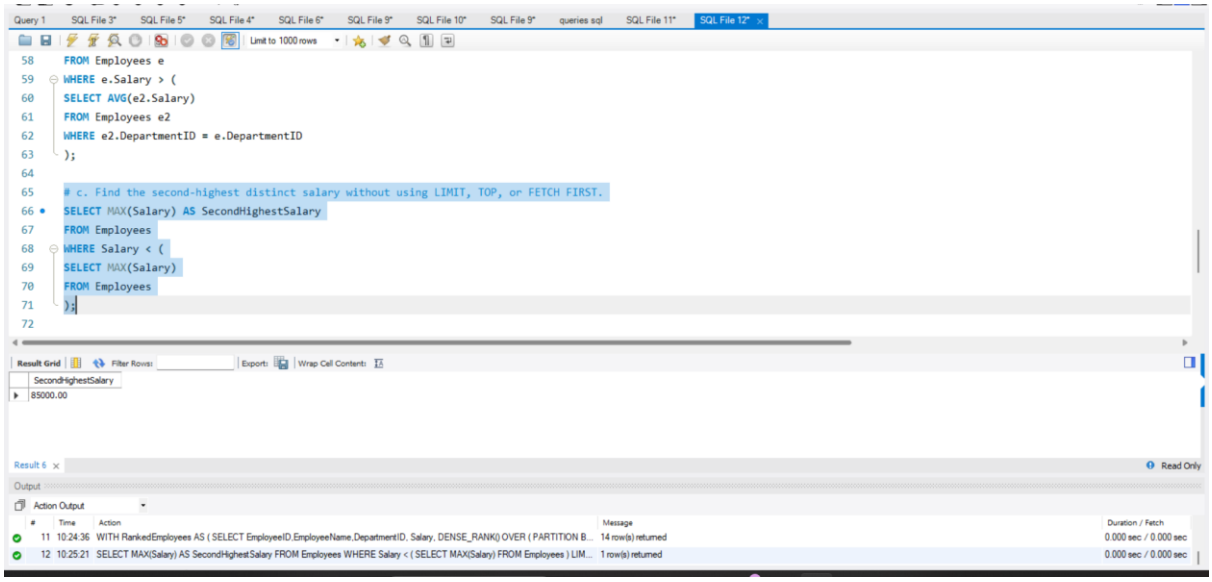
(a) Display employee, department and manager names



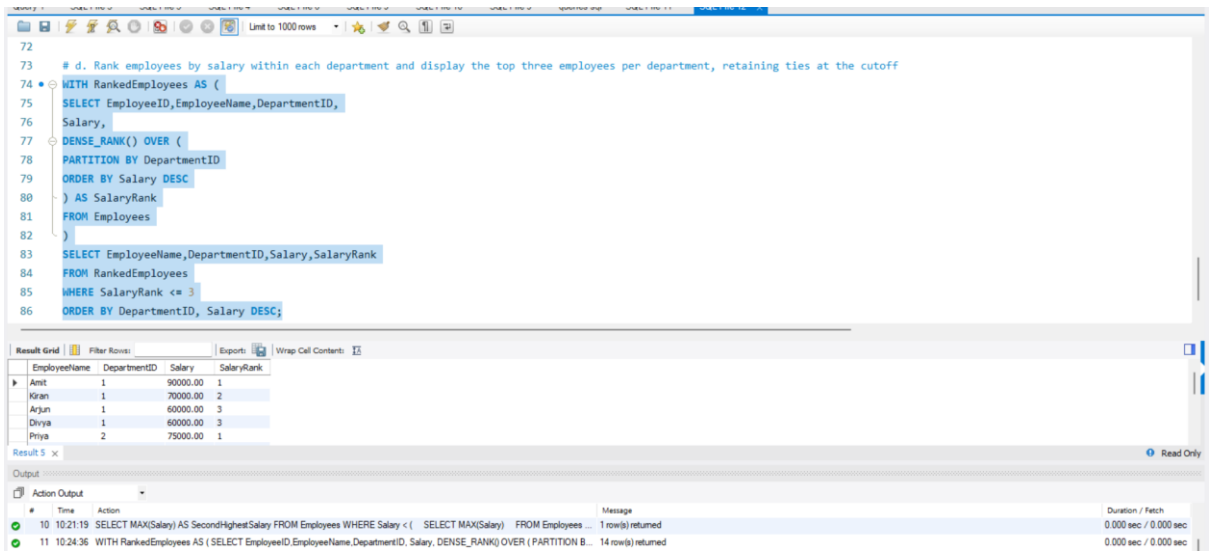
b. Find employees earning more than their department's average salary.



c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

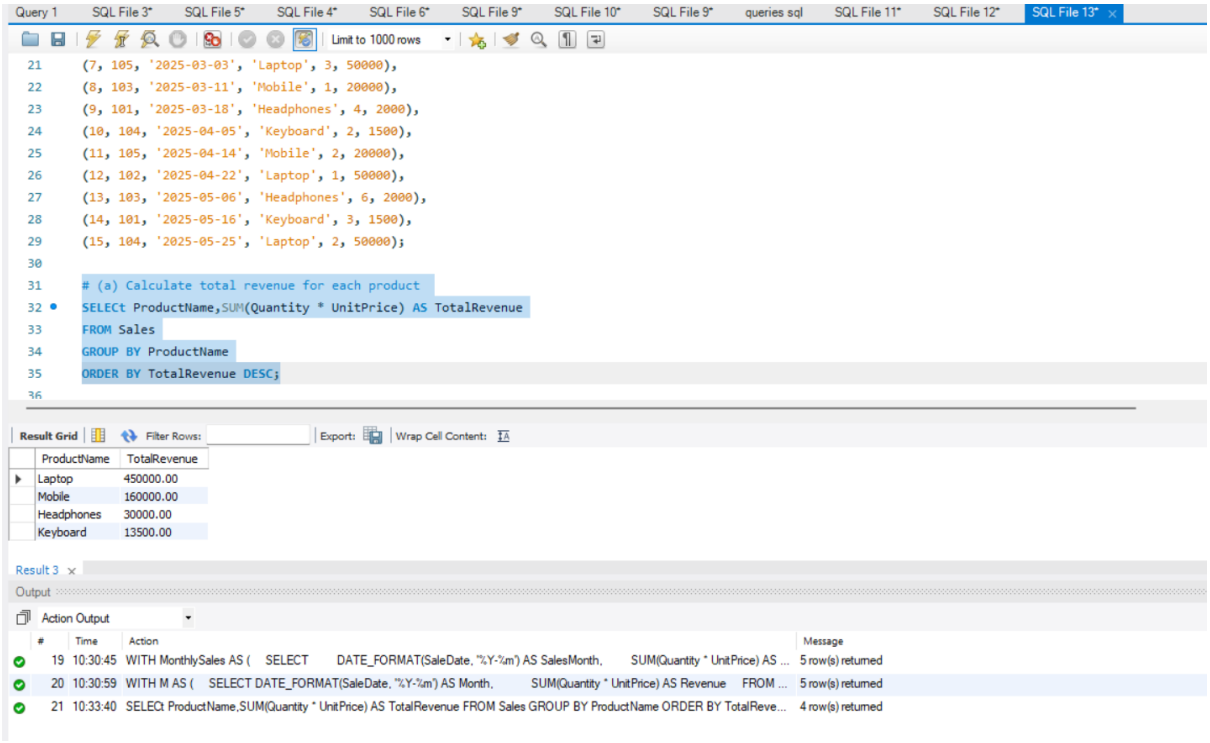


d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.



Question 2: Sales Analytics and Query Optimization

a. Calculate total revenue for each product.



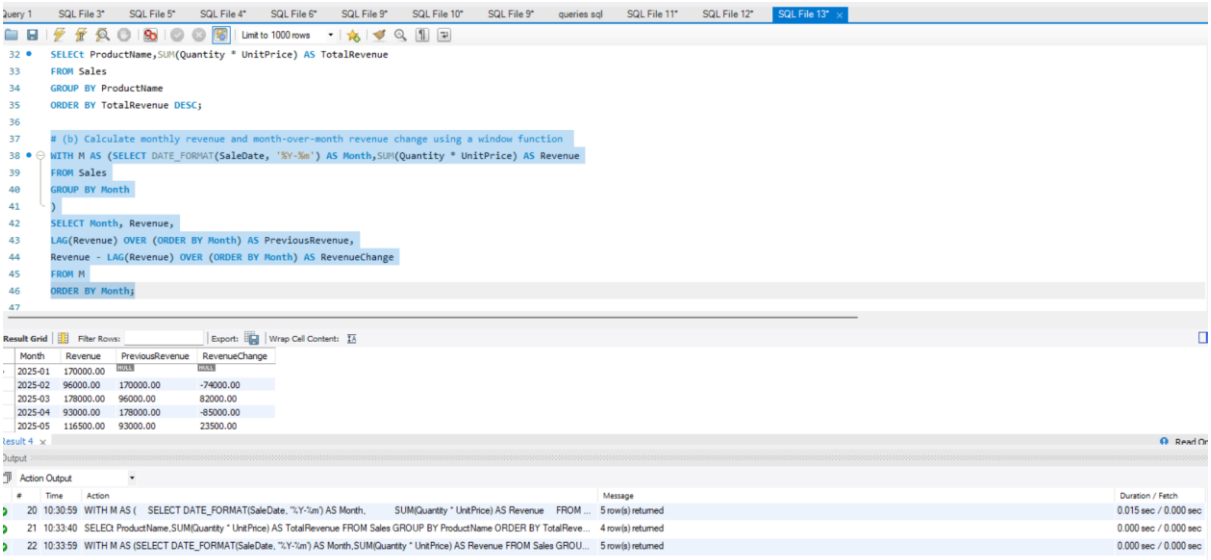
```
(7, 105, '2025-03-03', 'Laptop', 3, 50000),
(8, 103, '2025-03-11', 'Mobile', 1, 20000),
(9, 101, '2025-03-18', 'Headphones', 4, 2000),
(10, 104, '2025-04-05', 'Keyboard', 2, 1500),
(11, 105, '2025-04-14', 'Mobile', 2, 20000),
(12, 102, '2025-04-22', 'Laptop', 1, 50000),
(13, 103, '2025-05-06', 'Headphones', 6, 2000),
(14, 101, '2025-05-16', 'Keyboard', 3, 1500),
(15, 104, '2025-05-25', 'Laptop', 2, 50000);

# (a) Calculate total revenue for each product
SELECt ProductName,SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName
ORDER BY TotalRevenue DESC;
```

ProductName	TotalRevenue
Laptop	450000.00
Mobile	160000.00
Headphones	30000.00
Keyboard	13500.00

```
19 10:30:45 WITH MonthlySales AS ( SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS SalesMonth, SUM(Quantity * UnitPrice) AS ... 5 row(s) returned
20 10:30:59 WITH M AS ( SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month, SUM(Quantity * UnitPrice) AS Revenue FROM ... 5 row(s) returned
21 10:33:40 SELECt ProductName,SUM(Quantity * UnitPrice) AS TotalRevenue FROM Sales GROUP BY ProductName ORDER BY TotalReve... 4 row(s) returned
```

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)



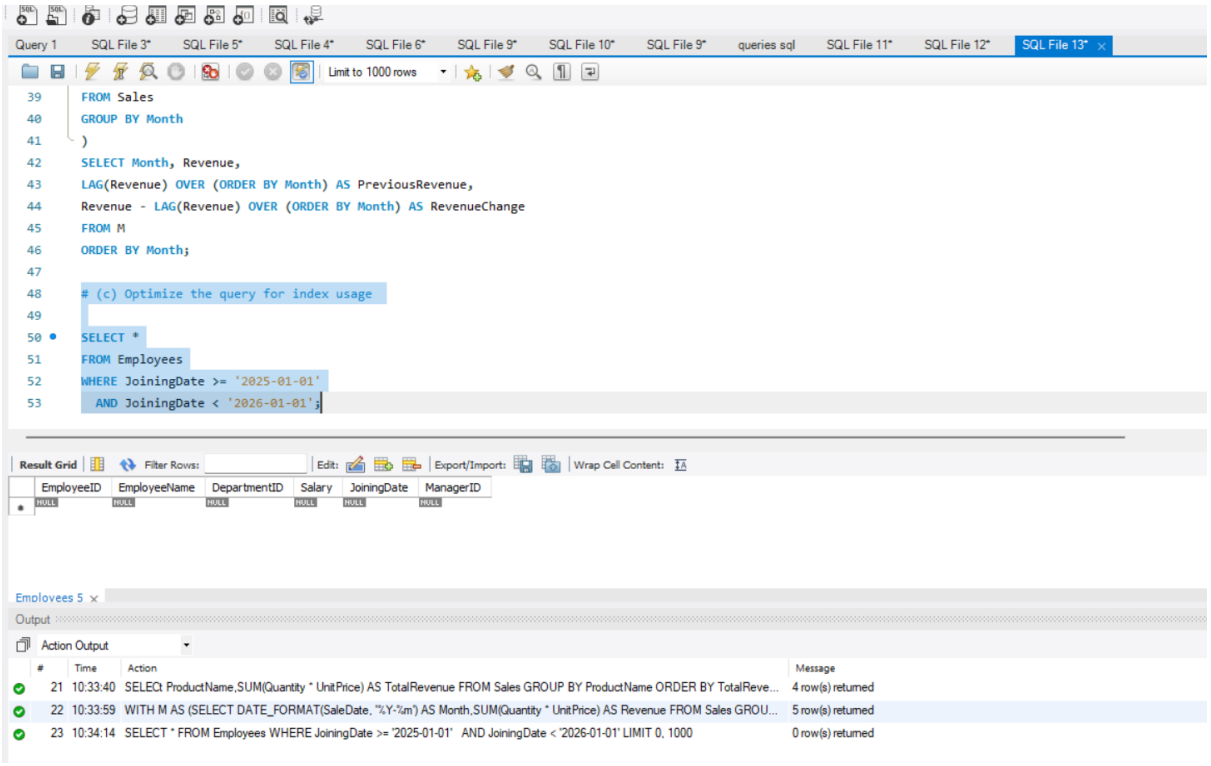
```
SELECt ProductName,SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName
ORDER BY TotalRevenue DESC;

# (b) Calculate monthly revenue and month-over-month revenue change using a window function
WITH M AS (SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month,SUM(Quantity * UnitPrice) AS Revenue
FROM Sales
GROUP BY Month
)
SELECT Month, Revenue,
LAG(Revenue) OVER (ORDER BY Month) AS PreviousRevenue,
Revenue - LAG(Revenue) OVER (ORDER BY Month) AS RevenueChange
FROM M
ORDER BY Month;
```

Month	Revenue	PreviousRevenue	RevenueChange
2025-01	170000.00	NULL	NULL
2025-02	96000.00	170000.00	-74000.00
2025-03	178000.00	96000.00	82000.00
2025-04	93000.00	178000.00	-85000.00
2025-05	116500.00	93000.00	23500.00

```
20 10:30:59 WITH M AS ( SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month, SUM(Quantity * UnitPrice) AS Revenue FROM ... 5 row(s) returned 0.015 sec / 0.000 sec
21 10:33:40 SELECt ProductName,SUM(Quantity * UnitPrice) AS TotalRevenue FROM Sales GROUP BY ProductName ORDER BY TotalReve... 4 row(s) returned 0.000 sec / 0.000 sec
22 10:33:59 WITH M AS (SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month,SUM(Quantity * UnitPrice) AS Revenue FROM Sales GROU... 5 row(s) returned 0.000 sec / 0.000 sec
```

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better.



Question 3: SQL Troubleshooting

(a) Explain how an execution plan helps identify expensive operations in a query.

An execution plan shows how the database executes a SQL query and helps identify performance.

- It provides estimated or actual row counts and costs to help optimize the query.

Ex:

EXPLAIN

SELECT *FROM Employees

WHERE DepartmentID = 1;

Explanation: EXPLAIN shows how MySQL plans to execute the query and whether an index can be used.

(b) Recommend an appropriate index strategy for DepartmentID and Salary.

EX:

CREATE INDEX idx_dept_salary

ON Employees (DepartmentID, Salary);

Quary :

SELECT *FROM Employees

WHERE DepartmentID = 1

AND Salary > 50000;

- DepartmentID is the first column, so the index supports filtering by department.
- Salary is the second column, supporting salary range filtering within a department.
- A composite index can reduce the no of rows examined and improve query performance.

(c) Identify duplicate customer email addresses occurring more than once.

SELECT Email, COUNT(*) AS Total

FROM Customers

GROUP BY Email

HAVING COUNT(*) > 1;

- GROUP BY Email groups records with the same email address.
- COUNT(*) counts occurrences of each address.
- HAVING COUNT(*) > 1 displays only duplicate email addresses.

SECTION B: LINUX

Question 4: Log Analysis

(a) Find all lines containing ERROR:

grep "ERROR" /var/log/application.log

(b) Display the 20 most recent error entries

grep "ERROR" /var/log/application.log | tail -n 20

(c) Count the total number of error entries

grep -c "ERROR" /var/log/application.log

(d) Extract error entries for a specified date into a separate file

grep "2026-10-28" /var/log/application.log | grep "ERROR" > errors_2026-10-28.log

(e) Calculate the number of errors occurring in each hour

grep "ERROR" /var/log/application.log |

awk '{print substr($2,1,2)}' |

sort | uniq -c

- awk extracts the hour from the timestamp (for ex, 10 from 10:15:23).
- sort groups identical hours together.
- uniq -c counts the errors for each hour.

Question 5: Permissions and System Troubleshooting

a) Create a group named finance and a report file

sudo groupadd finance

sudo touch financial_report.txt

- roupadd creates the group, and touch creates the report file.

(b) Set the required permissions

CMD :

sudo chown "$USER":finance financial_report.txt

chmod 640 financial_report.txt

(c) Identify processes consuming the most CPU

CMD : ps aux --sort=-%cpu | head

(d) Display memory usage and filesystem disk utilization

free -h : displays memory usage

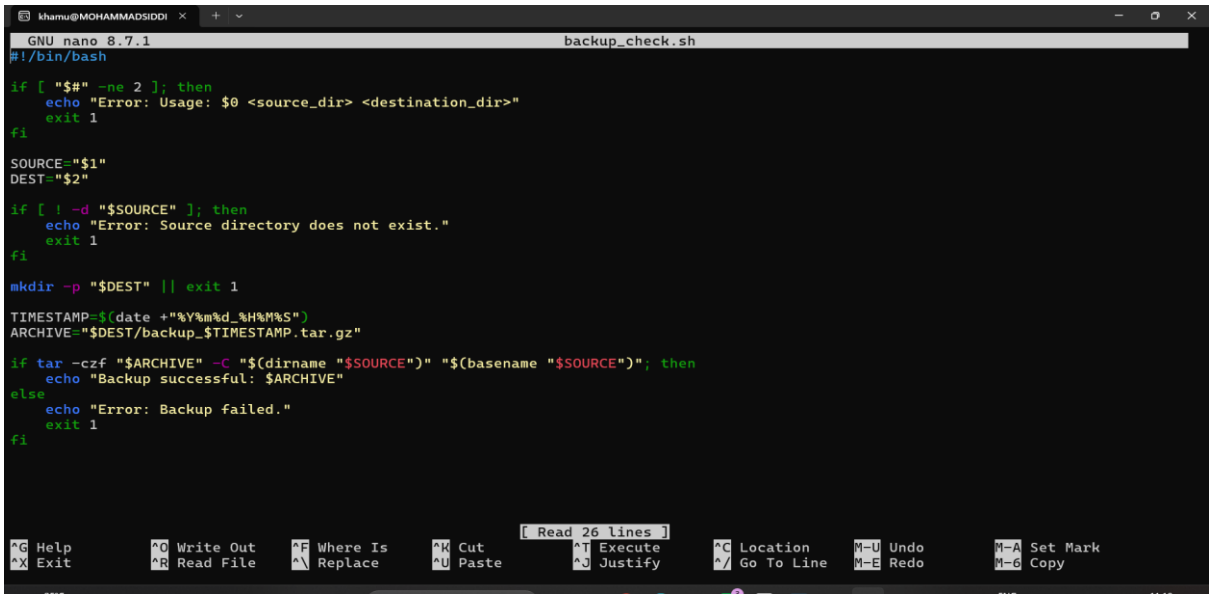
df -h : displays filesystem disk space

(e) Find files larger than 100 MB under an application directory

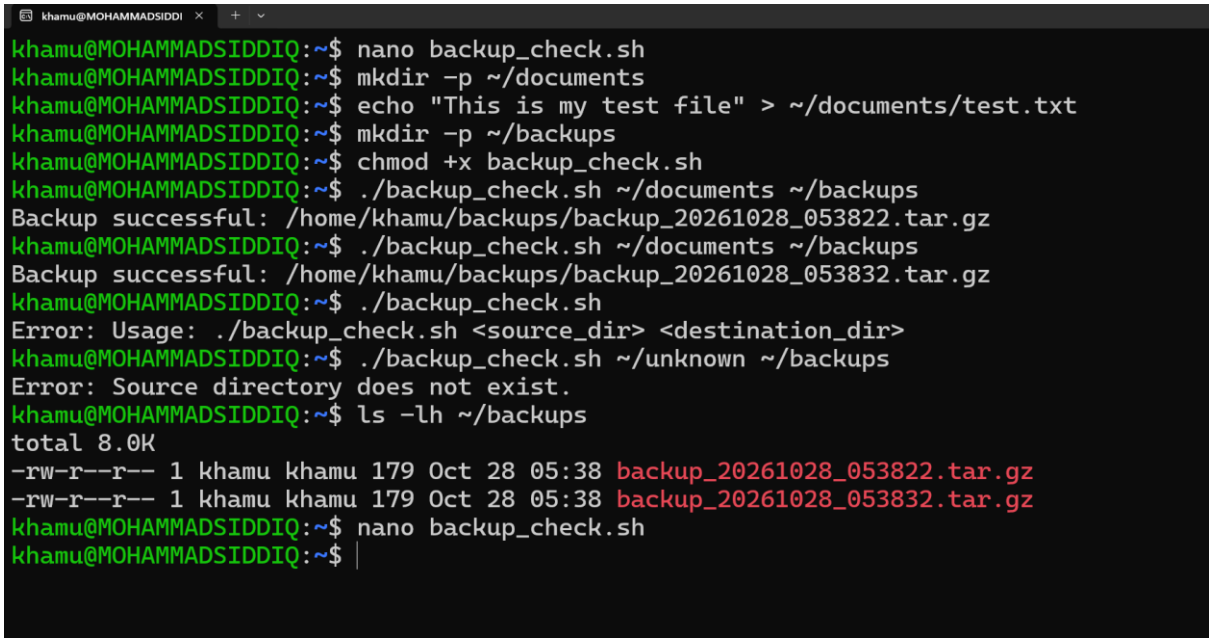
CMD : find /var/www/app -type f -size +100M

Question 6: Bash Scripting

Create the script file : nano backup_check.sh



UBUNTU TERMINAL OUTPUTS:



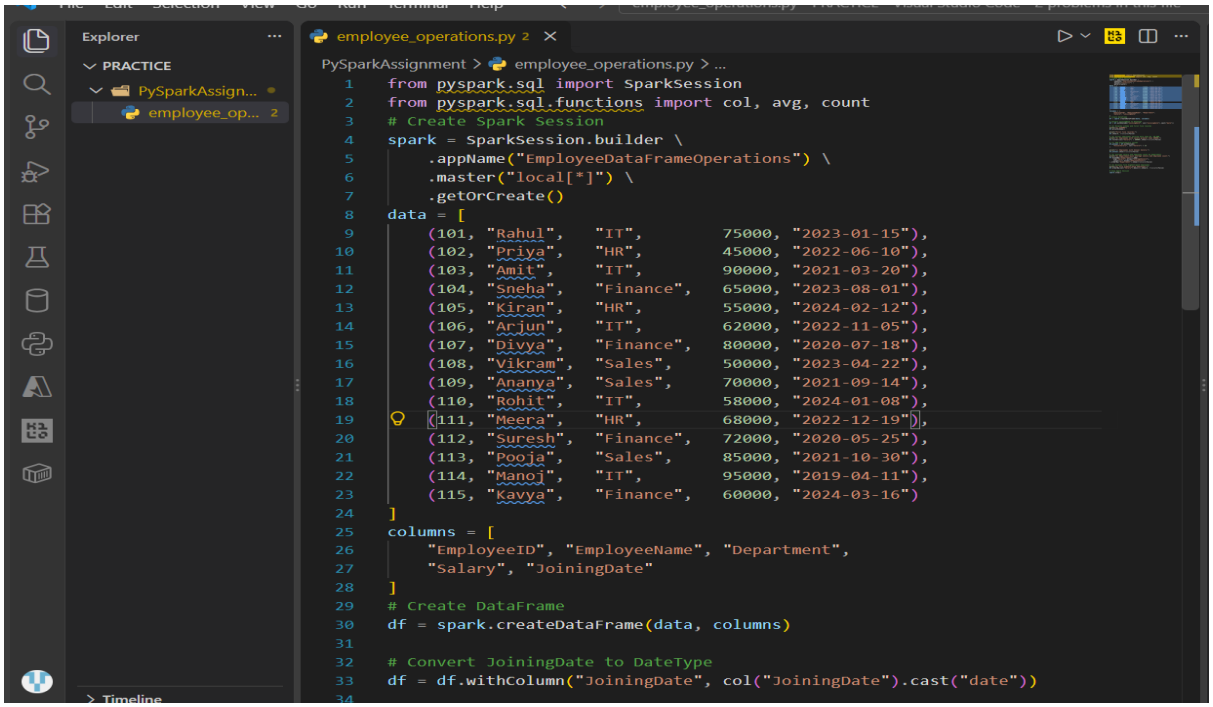
Output: Backup successful

Error: Usage

SECTION C: PYSPARK DATA ENGINEERING

Question 7: PySpark DataFrame Operations

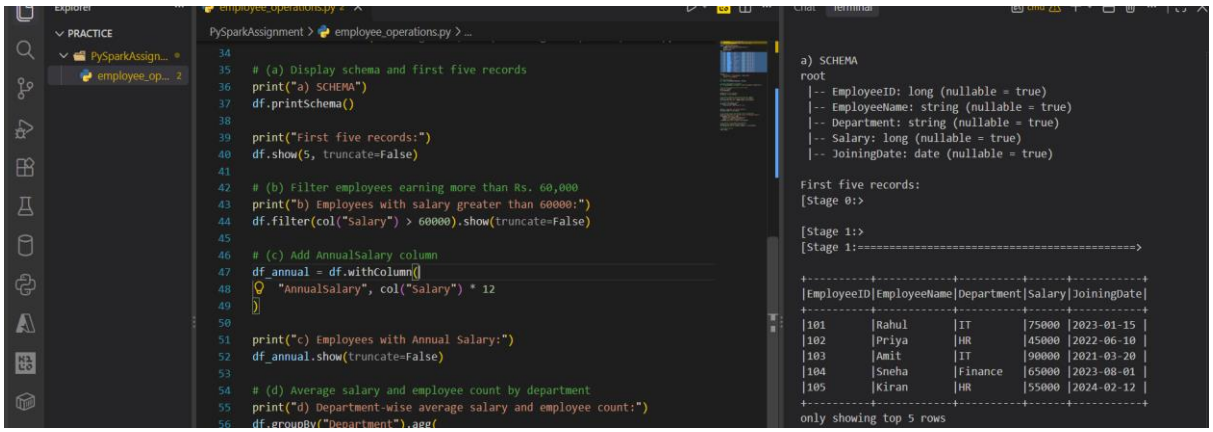
Create a PySpark DataFrame



```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count
# Create Spark Session
spark = SparkSession.builder \
    .appName("EmployeeDataFrameOperations") \
    .master("local[*]") \
    .getOrCreate()
data = [
    (101, "Rahul",   "IT",        75000, "2023-01-15"),
    (102, "Priya",   "HR",        45000, "2022-06-10"),
    (103, "Amit",    "IT",        90000, "2021-03-20"),
    (104, "Sneha",   "Finance",   65000, "2023-08-01"),
    (105, "Kiran",   "HR",        55000, "2024-02-12"),
    (106, "Arjun",   "IT",        62000, "2022-11-05"),
    (107, "Divya",   "Finance",   80000, "2020-07-18"),
    (108, "Vikram",  "Sales",     50000, "2023-04-22"),
    (109, "Ananya",  "Sales",     70000, "2021-09-14"),
    (110, "Rohit",   "IT",        58000, "2024-01-08"),
    (111, "Meera",   "HR",        68000, "2022-12-19"),
    (112, "Suresh",  "Finance",   72000, "2020-05-25"),
    (113, "Pooja",   "Sales",     85000, "2021-10-30"),
    (114, "Manoj",   "IT",        95000, "2019-04-11"),
    (115, "Kavya",   "Finance",   60000, "2024-03-16")
]
columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]
# Create DataFrame
df = spark.createDataFrame(data, columns)

# Convert JoiningDate to DateType
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))
```

a. Display the schema and first five records.



```python
# (a) Display schema and first five records
print("a) SCHEMA")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)

# (b) Filter employees earning more than Rs. 60,000
print("b) Employees with salary greater than 60000:")
df.filter(col("Salary") > 60000).show(truncate=False)

# (c) Add AnnualSalary column
df_annual = df.withColumn(
    "AnnualSalary", col("Salary") * 12
)

print("c) Employees with Annual Salary:")
df_annual.show(truncate=False)

# (d) Average salary and employee count by department
print("d) Department-wise average salary and employee count:")
```

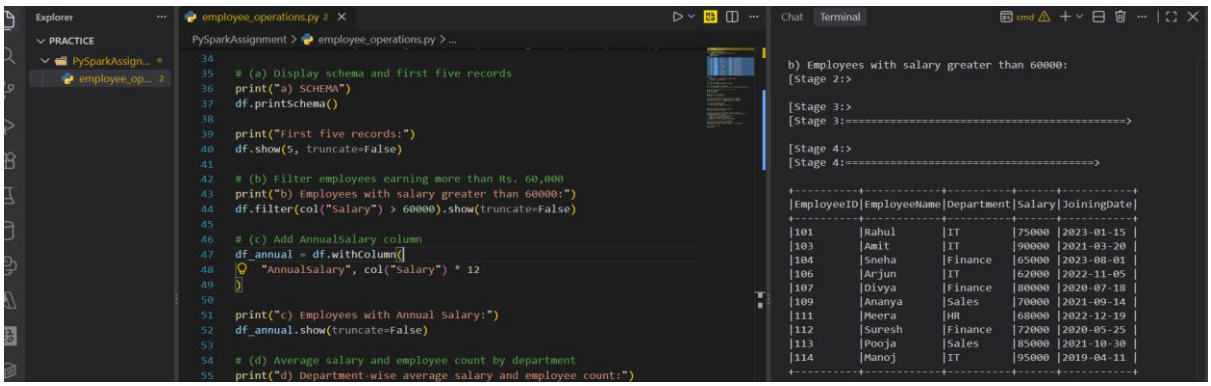
```
a) SCHEMA
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

First five records:
[Stage 0:>

[Stage 1:>
[Stage 1:=============================================>

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Rahul       |IT        |75000 |2023-01-15 |
|102       |Priya       |HR        |45000 |2022-06-10 |
|103       |Amit        |IT        |90000 |2021-03-20 |
|104       |Sneha       |Finance   |65000 |2023-08-01 |
|105       |Kiran       |HR        |55000 |2024-02-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000.



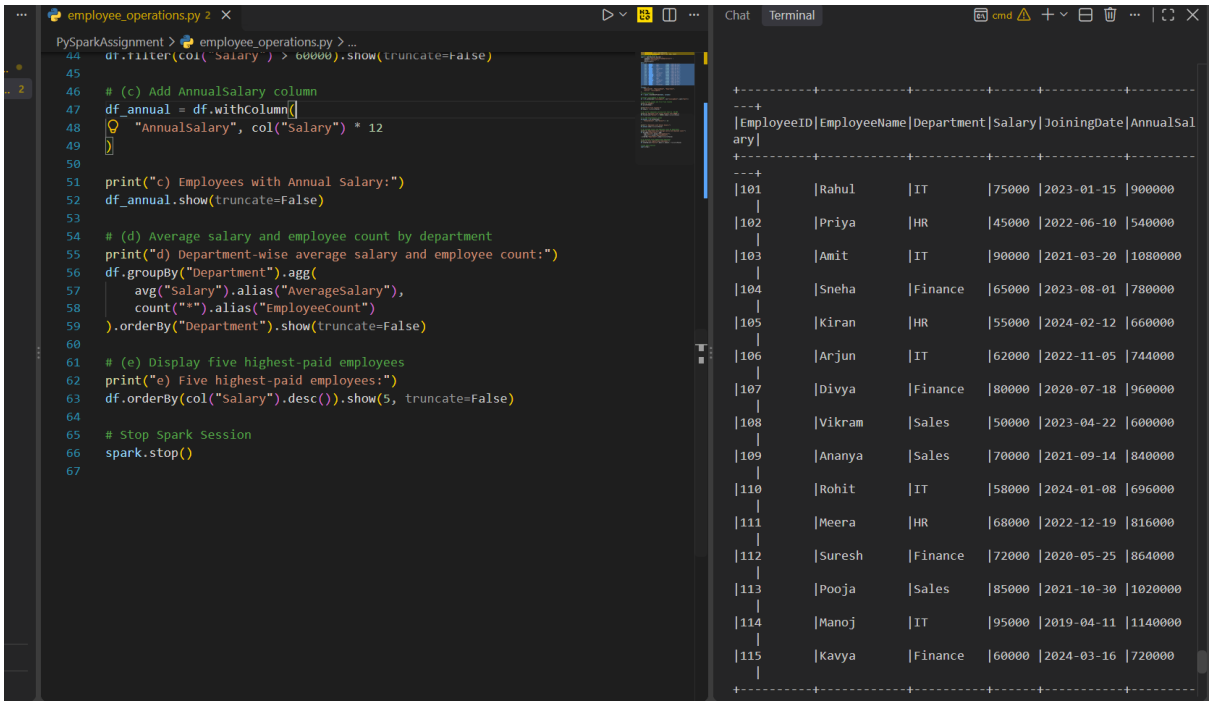
```
b) Employees with salary greater than 60000:
[Stage 2:>

[Stage 3:>
[Stage 3:=============================================>

[Stage 4:>
[Stage 4:=====================================>

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Rahul       |IT        |75000 |2023-01-15 |
|103       |Amit        |IT        |90000 |2021-03-20 |
|104       |Sneha       |Finance   |65000 |2023-08-01 |
|106       |Arjun       |IT        |62000 |2022-11-05 |
|107       |Divya       |Finance   |80000 |2020-07-18 |
|109       |Ananya      |Sales     |70000 |2021-09-14 |
|111       |Meera       |HR        |68000 |2022-12-19 |
|112       |Suresh      |Finance   |72000 |2020-05-25 |
|113       |Pooja       |Sales     |85000 |2021-10-30 |
|114       |Manoj       |IT        |95000 |2019-04-11 |
+----------+------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary



```
df.filter(col("Salary") > 60000).show(truncate=False)

# (c) Add AnnualSalary column
df_annual = df.withColumn(
    "AnnualSalary", col("Salary") * 12
)

print("c) Employees with Annual Salary:")
df_annual.show(truncate=False)

# (d) Average salary and employee count by department
print("d) Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).orderBy("Department").show(truncate=False)

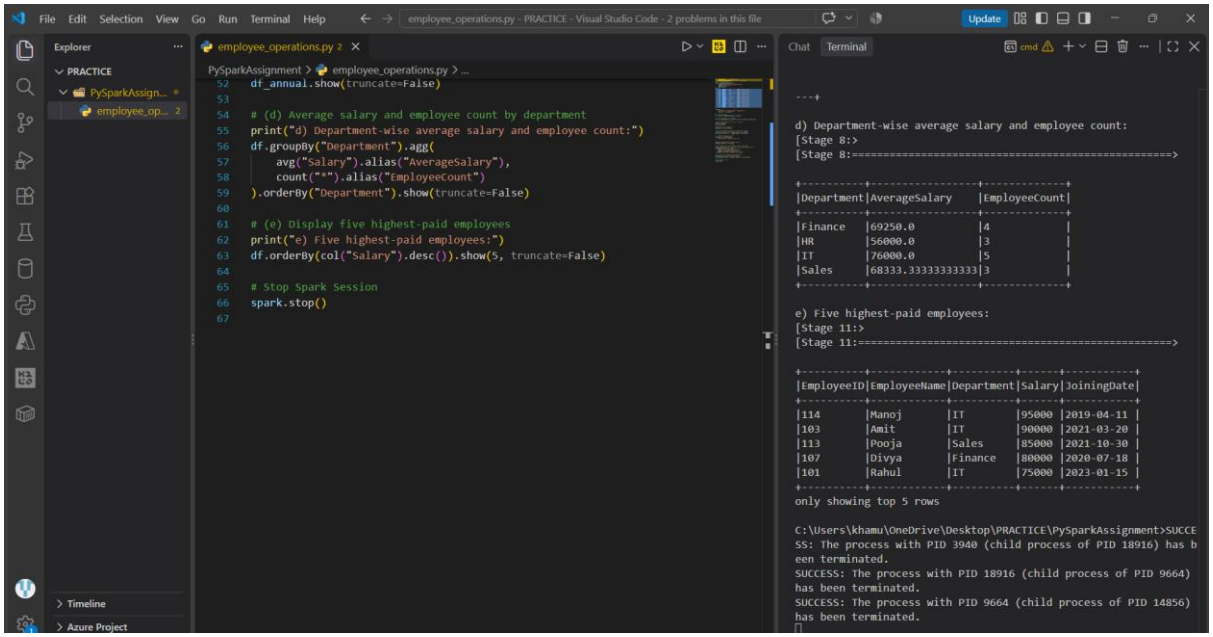
# (e) Display five highest-paid employees
print("e) Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5, truncate=False)

# Stop Spark Session
spark.stop()
```

```
+----------+------------+----------+------+-----------+---------
---+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSal
ary|
+----------+------------+----------+------+-----------+---------
---+
|101       |Rahul       |IT        |75000 |2023-01-15 |900000
   |
|102       |Priya       |HR        |45000 |2022-06-10 |540000
   |
|103       |Amit        |IT        |90000 |2021-03-20 |1080000
   |
|104       |Sneha       |Finance   |65000 |2023-08-01 |780000
   |
|105       |Kiran       |HR        |55000 |2024-02-12 |660000
   |
|106       |Arjun       |IT        |62000 |2022-11-05 |744000
   |
|107       |Divya       |Finance   |80000 |2020-07-18 |960000
   |
|108       |Vikram      |Sales     |50000 |2023-04-22 |600000
   |
|109       |Ananya      |Sales     |70000 |2021-09-14 |840000
   |
|110       |Rohit       |IT        |58000 |2024-01-08 |696000
   |
|111       |Meera       |HR        |68000 |2022-12-19 |816000
   |
|112       |Suresh      |Finance   |72000 |2020-05-25 |864000
   |
|113       |Pooja       |Sales     |85000 |2021-10-30 |1020000
   |
|114       |Manoj       |IT        |95000 |2019-04-11 |1140000
   |
|115       |Kavya       |Finance   |60000 |2024-03-16 |720000
   |
+----------+------------+----------+------+-----------+---------
```

d. Group employees by department and calculate the average salary and employee count. &

e. Display the five highest-paid employees.



```
df_annual.show(truncate=False)

# (d) Average salary and employee count by department
print("d) Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).orderBy("Department").show(truncate=False)

# (e) Display five highest-paid employees
print("e) Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5, truncate=False)

# Stop Spark Session
spark.stop()
```

```
---+

d) Department-wise average salary and employee count:
[Stage 8:>
[Stage 8:==================================================>

+----------+-----------------+-------------+
|Department|AverageSalary    |EmployeeCount|
+----------+-----------------+-------------+
|Finance   |69250.0          |4            |
|HR        |56000.0          |3            |
|IT        |76000.0          |5            |
|Sales     |68333.33333333333|3            |
+----------+-----------------+-------------+

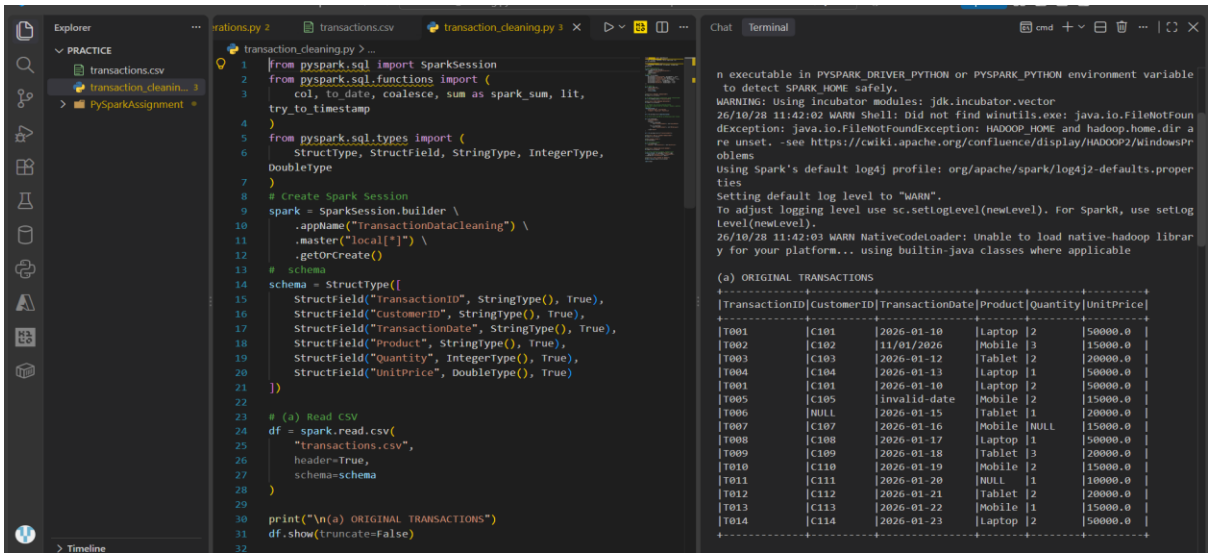
e) Five highest-paid employees:
[Stage 11:>
[Stage 11:==================================================>

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|114       |Manoj       |IT        |95000 |2019-04-11 |
|103       |Amit        |IT        |90000 |2021-03-20 |
|113       |Pooja       |Sales     |85000 |2021-10-30 |
|107       |Divya       |Finance   |80000 |2020-07-18 |
|101       |Rahul       |IT        |75000 |2023-01-15 |
+----------+------------+----------+------+-----------+
only showing top 5 rows

C:\Users\khamu\OneDrive\Desktop\PRACTICE\PySparkAssignment>SUCCE
SS: The process with PID 3940 (child process of PID 18916) has b
een terminated.
SUCCESS: The process with PID 18916 (child process of PID 9664)
has been terminated.
SUCCESS: The process with PID 9664 (child process of PID 14856)
has been terminated.
```

Question 8: Data Cleaning and Aggregation

a. Read the CSV file using an appropriate schema.



```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, to_date, coalesce, sum as spark_sum, lit,
try_to_timestamp
)
from pyspark.sql.types import (
    StructType, StructField, StringType, IntegerType,
DoubleType
)
# Create Spark Session
spark = SparkSession.builder \
    .appName("TransactionDataCleaning") \
    .master("local[*]") \
    .getOrCreate()
#  schema
schema = StructType([
    StructField("TransactionID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("TransactionDate", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True)
])

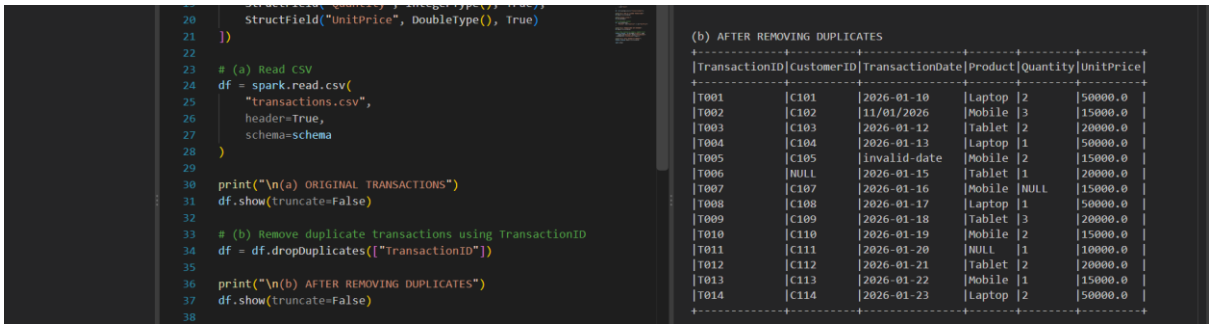
# (a) Read CSV
df = spark.read.csv(
    "transactions.csv",
    header=True,
    schema=schema
)

print("\n(a) ORIGINAL TRANSACTIONS")
df.show(truncate=False)
```

```
n executable in PYSPARK_DRIVER_PYTHON or PYSPARK_PYTHON environment variable
 to detect SPARK_HOME safely.
WARNING: Using incubator modules: jdk.incubator.vector
26/10/28 11:42:02 WARN Shell: Did not find winutils.exe: java.io.FileNotFoun
dException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir a
re unset. -see https://cwiki.apache.org/confluence/display/HADOOP2/WindowsPr
oblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.proper
ties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLog
Level(newLevel).
26/10/28 11:42:03 WARN NativeCodeLoader: Unable to load native-hadoop librar
y for your platform... using builtin-java classes where applicable

(a) ORIGINAL TRANSACTIONS
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|T001         |C101      |2026-01-10     |Laptop |2       |50000.0  |
|T002         |C102      |11/01/2026     |Mobile |3       |15000.0  |
|T003         |C103      |2026-01-12     |Tablet |2       |20000.0  |
|T004         |C104      |2026-01-13     |Laptop |1       |50000.0  |
|T001         |C101      |2026-01-10     |Laptop |2       |50000.0  |
|T005         |C105      |invalid-date   |Mobile |2       |15000.0  |
|T006         |NULL      |2026-01-15     |Tablet |1       |20000.0  |
|T007         |C107      |2026-01-16     |Mobile |NULL    |15000.0  |
|T008         |C108      |2026-01-17     |Laptop |1       |50000.0  |
|T009         |C109      |2026-01-18     |Tablet |3       |20000.0  |
|T010         |C110      |2026-01-19     |Mobile |2       |15000.0  |
|T011         |C111      |2026-01-20     |NULL   |1       |10000.0  |
|T012         |C112      |2026-01-21     |Tablet |2       |20000.0  |
|T013         |C113      |2026-01-22     |Mobile |1       |15000.0  |
|T014         |C114      |2026-01-23     |Laptop |2       |50000.0  |
+-------------+----------+---------------+-------+--------+---------+
```

b. Identify and remove duplicate transactions using a suitable business key



```
    StructField("UnitPrice", DoubleType(), True)
])

# (a) Read CSV
df = spark.read.csv(
    "transactions.csv",
    header=True,
    schema=schema
)

print("\n(a) ORIGINAL TRANSACTIONS")
df.show(truncate=False)

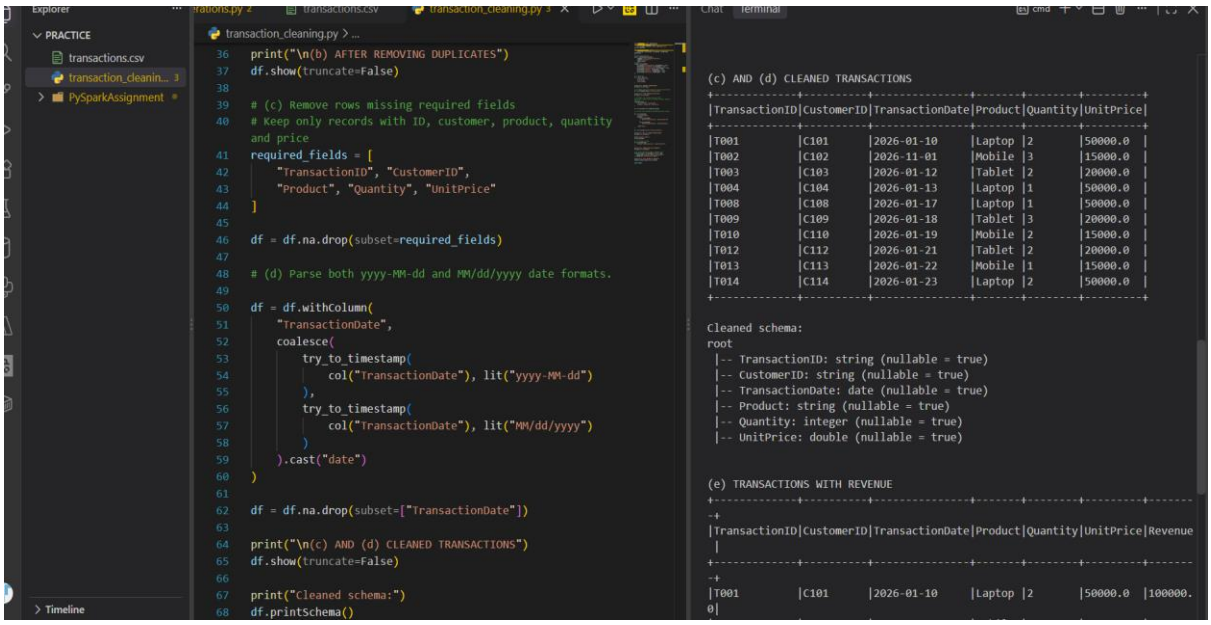
# (b) Remove duplicate transactions using TransactionID
df = df.dropDuplicates(["TransactionID"])

print("\n(b) AFTER REMOVING DUPLICATES")
df.show(truncate=False)
```

```
(b) AFTER REMOVING DUPLICATES
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|T001         |C101      |2026-01-10     |Laptop |2       |50000.0  |
|T002         |C102      |11/01/2026     |Mobile |3       |15000.0  |
|T003         |C103      |2026-01-12     |Tablet |2       |20000.0  |
|T004         |C104      |2026-01-13     |Laptop |1       |50000.0  |
|T005         |C105      |invalid-date   |Mobile |2       |15000.0  |
|T006         |NULL      |2026-01-15     |Tablet |1       |20000.0  |
|T007         |C107      |2026-01-16     |Mobile |NULL    |15000.0  |
|T008         |C108      |2026-01-17     |Laptop |1       |50000.0  |
|T009         |C109      |2026-01-18     |Tablet |3       |20000.0  |
|T010         |C110      |2026-01-19     |Mobile |2       |15000.0  |
|T011         |C111      |2026-01-20     |NULL   |1       |10000.0  |
|T012         |C112      |2026-01-21     |Tablet |2       |20000.0  |
|T013         |C113      |2026-01-22     |Mobile |1       |15000.0  |
|T014         |C114      |2026-01-23     |Laptop |2       |50000.0  |
+-------------+----------+---------------+-------+--------+---------+
```

c. Handle missing values in required fields and explain your chosen approach. &

d. Convert TransactionDate into a date type and handle invalid dates.



```
print("\n(b) AFTER REMOVING DUPLICATES")
df.show(truncate=False)

# (c) Remove rows missing required fields
# Keep only records with ID, customer, product, quantity
and price
required_fields = [
    "TransactionID", "CustomerID",
    "Product", "Quantity", "UnitPrice"
]

df = df.na.drop(subset=required_fields)

# (d) Parse both yyyy-MM-dd and MM/dd/yyyy date formats.

df = df.withColumn(
    "TransactionDate",
    coalesce(
        try_to_timestamp(
            col("TransactionDate"), lit("yyyy-MM-dd")
        ),
        try_to_timestamp(
            col("TransactionDate"), lit("MM/dd/yyyy")
        )
    ).cast("date")
)

df = df.na.drop(subset=["TransactionDate"])

print("\n(c) AND (d) CLEANED TRANSACTIONS")
df.show(truncate=False)

print("Cleaned schema:")
df.printSchema()
```

```
(c) AND (d) CLEANED TRANSACTIONS
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|T001         |C101      |2026-01-10     |Laptop |2       |50000.0  |
|T002         |C102      |2026-11-01     |Mobile |3       |15000.0  |
|T003         |C103      |2026-01-12     |Tablet |2       |20000.0  |
|T004         |C104      |2026-01-13     |Laptop |1       |50000.0  |
|T008         |C108      |2026-01-17     |Laptop |1       |50000.0  |
|T009         |C109      |2026-01-18     |Tablet |3       |20000.0  |
|T010         |C110      |2026-01-19     |Mobile |2       |15000.0  |
|T012         |C112      |2026-01-21     |Tablet |2       |20000.0  |
|T013         |C113      |2026-01-22     |Mobile |1       |15000.0  |
|T014         |C114      |2026-01-23     |Laptop |2       |50000.0  |
+-------------+----------+---------------+-------+--------+---------+

Cleaned schema:
root
 |-- TransactionID: string (nullable = true)
 |-- CustomerID: string (nullable = true)
 |-- TransactionDate: date (nullable = true)
 |-- Product: string (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- UnitPrice: double (nullable = true)


(e) TRANSACTIONS WITH REVENUE
+-------------+----------+---------------+-------+--------+---------+-------
-+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue
 |
+-------------+----------+---------------+-------+--------+---------+-------
-+
|T001         |C101      |2026-01-10     |Laptop |2       |50000.0  |100000.
0|
```

e. Calculate revenue using Quantity × UnitPrice

```
            try_to_timestamp(
                col("TransactionDate"), lit("yyyy-MM-dd")
            ),
            try_to_timestamp(
                col("TransactionDate"), lit("MM/dd/yyyy")
            )
        ).cast("date")
    )

    df = df.na.drop(subset=["TransactionDate"])

    print("\n(c) AND (d) CLEANED TRANSACTIONS")
    df.show(truncate=False)

    print("Cleaned schema:")
    df.printSchema()

    # (e) Calculate revenue
    df = df.withColumn(
        "Revenue", col("Quantity") * col("UnitPrice")
    )

    print("\n(e) TRANSACTIONS WITH REVENUE")
    df.show(truncate=False)

    # (f) Total revenue by product, highest first
    product_revenue = df.groupBy("Product").agg(
        spark_sum("Revenue").alias("TotalRevenue")
```

```
 |-- UnitPrice: double (nullable = true)


(e) TRANSACTIONS WITH REVENUE
+-------------+----------+---------------+-------+--------+---------+-------
-+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue
 |
+-------------+----------+---------------+-------+--------+---------+-------
-+
|T001         |C101      |2026-01-10     |Laptop |2       |50000.0  |100000.
0|
|T002         |C102      |2026-11-01     |Mobile |3       |15000.0  |45000.0
 |
|T003         |C103      |2026-01-12     |Tablet |2       |20000.0  |40000.0
 |
|T004         |C104      |2026-01-13     |Laptop |1       |50000.0  |50000.0
 |
|T008         |C108      |2026-01-17     |Laptop |1       |50000.0  |50000.0
 |
|T009         |C109      |2026-01-18     |Tablet |3       |20000.0  |60000.0
 |
|T010         |C110      |2026-01-19     |Mobile |2       |15000.0  |30000.0
 |
|T012         |C112      |2026-01-21     |Tablet |2       |20000.0  |40000.0
 |
|T013         |C113      |2026-01-22     |Mobile |1       |15000.0  |15000.0
 |
|T014         |C114      |2026-01-23     |Laptop |2       |50000.0  |100000.
0|
+-------------+----------+---------------+-------+--------+---------+-------
```

f. Calculate total revenue by product and display the results in descending order.

```
# (f) Total revenue by product, highest first
product_revenue = df.groupBy("Product").agg(
    spark_sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc())

print("\n(f) TOTAL REVENUE BY PRODUCT")
product_revenue.show(truncate=False)

spark.stop()
```

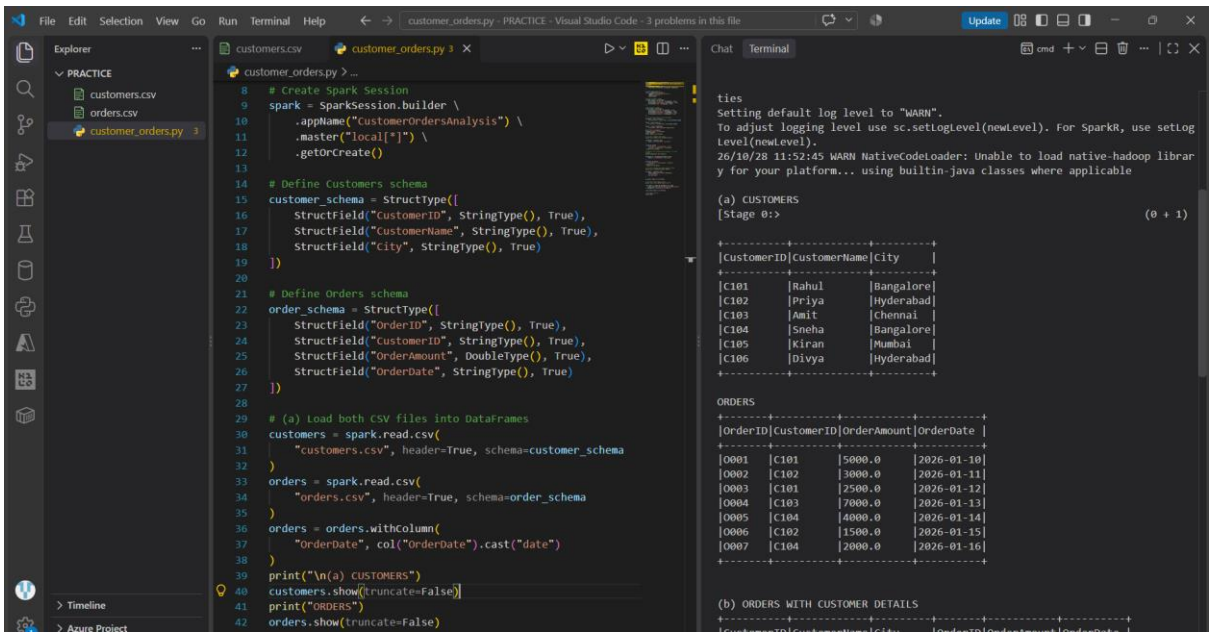
```
|T004         |C104      |2026-01-13     |Laptop |1       |50000.0  |50000.0
 |
|T008         |C108      |2026-01-17     |Laptop |1       |50000.0  |50000.0
 |
|T009         |C109      |2026-01-18     |Tablet |3       |20000.0  |60000.0
 |
|T010         |C110      |2026-01-19     |Mobile |2       |15000.0  |30000.0
 |
|T012         |C112      |2026-01-21     |Tablet |2       |20000.0  |40000.0
 |
|T013         |C113      |2026-01-22     |Mobile |1       |15000.0  |15000.0
 |
|T014         |C114      |2026-01-23     |Laptop |2       |50000.0  |100000.
0|
+-------------+----------+---------------+-------+--------+---------+-------
-+


(f) TOTAL REVENUE BY PRODUCT
+-------+------------+
|Product|TotalRevenue|
+-------+------------+
|Laptop |300000.0    |
|Tablet |140000.0    |
|Mobile |90000.0     |
+-------+------------+


C:\Users\khamu\OneDrive\Desktop\PRACTICE>SUCCESS: The process with PID 20812
 (child process of PID 25324) has been terminated.
SUCCESS: The process with PID 25324 (child process of PID 19444) has been te
rminated.
SUCCESS: The process with PID 19444 (child process of PID 2988) has been ter
minated.
```

Question 9: PySpark Joins and Performance

a. Load both datasets into DataFrames.



```
# Create Spark Session
spark = SparkSession.builder \
    .appName("CustomerOrdersAnalysis") \
    .master("local[*]") \
    .getOrCreate()

# Define Customers schema
customer_schema = StructType([
    StructField("CustomerID", StringType(), True),
    StructField("CustomerName", StringType(), True),
    StructField("City", StringType(), True)
])

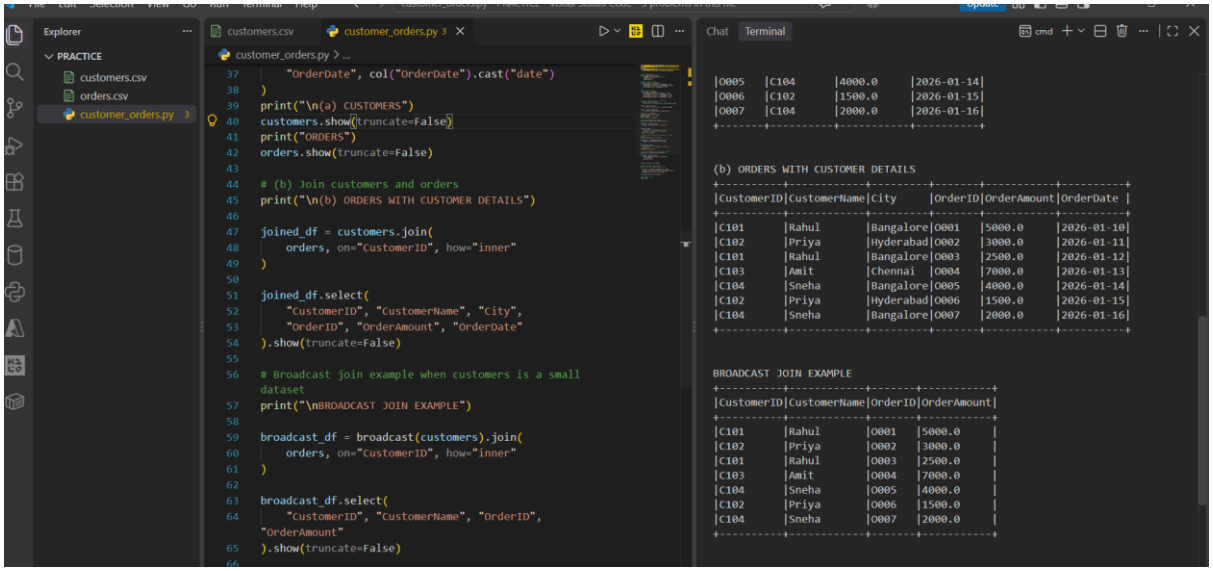
# Define Orders schema
order_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("OrderAmount", DoubleType(), True),
    StructField("OrderDate", StringType(), True)
])

# (a) Load both CSV files into DataFrames
customers = spark.read.csv(
    "customers.csv", header=True, schema=customer_schema
)
orders = spark.read.csv(
    "orders.csv", header=True, schema=order_schema
)
orders = orders.withColumn(
    "OrderDate", col("OrderDate").cast("date")
)
print("\n(a) CUSTOMERS")
customers.show(truncate=False)
print("ORDERS")
orders.show(truncate=False)
```

```
(a) CUSTOMERS
+----------+------------+---------+
|CustomerID|CustomerName|City     |
+----------+------------+---------+
|C101      |Rahul       |Bangalore|
|C102      |Priya       |Hyderabad|
|C103      |Amit        |Chennai  |
|C104      |Sneha       |Bangalore|
|C105      |Kiran       |Mumbai   |
|C106      |Divya       |Hyderabad|
+----------+------------+---------+

ORDERS
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount|OrderDate |
+-------+----------+-----------+----------+
|O001   |C101      |5000.0     |2026-01-10|
|O002   |C102      |3000.0     |2026-01-11|
|O003   |C101      |2500.0     |2026-01-12|
|O004   |C103      |7000.0     |2026-01-13|
|O005   |C104      |4000.0     |2026-01-14|
|O006   |C102      |1500.0     |2026-01-15|
|O007   |C104      |2000.0     |2026-01-16|
+-------+----------+-----------+----------+
```

b. Join the datasets to display orders with customer details



```
print("\n(b) ORDERS WITH CUSTOMER DETAILS")

joined_df = customers.join(
    orders, on="CustomerID", how="inner"
)

joined_df.select(
    "CustomerID", "CustomerName", "City",
    "OrderID", "OrderAmount", "OrderDate"
).show(truncate=False)

# Broadcast join example when customers is a small dataset
print("\nBROADCAST JOIN EXAMPLE")

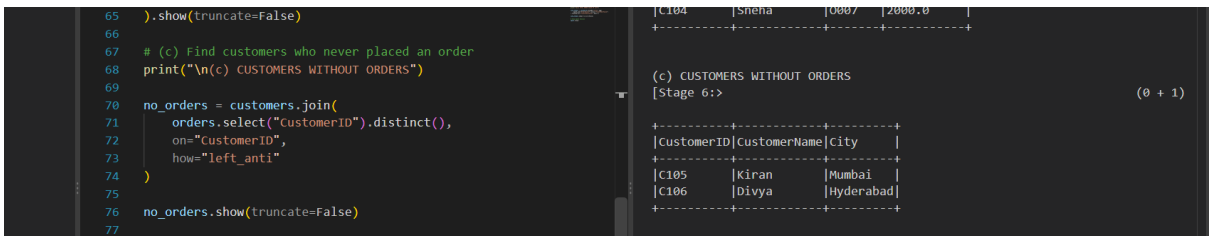
broadcast_df = broadcast(customers).join(
    orders, on="CustomerID", how="inner"
)

broadcast_df.select(
    "CustomerID", "CustomerName", "OrderID",
"OrderAmount"
).show(truncate=False)
```

```
(b) ORDERS WITH CUSTOMER DETAILS
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|City     |OrderID|OrderAmount|OrderDate |
+----------+------------+---------+-------+-----------+----------+
|C101      |Rahul       |Bangalore|O001   |5000.0     |2026-01-10|
|C102      |Priya       |Hyderabad|O002   |3000.0     |2026-01-11|
|C101      |Rahul       |Bangalore|O003   |2500.0     |2026-01-12|
|C103      |Amit        |Chennai  |O004   |7000.0     |2026-01-13|
|C104      |Sneha       |Bangalore|O005   |4000.0     |2026-01-14|
|C102      |Priya       |Hyderabad|O006   |1500.0     |2026-01-15|
|C104      |Sneha       |Bangalore|O007   |2000.0     |2026-01-16|
+----------+------------+---------+-------+-----------+----------+

BROADCAST JOIN EXAMPLE
+----------+------------+-------+-----------+
|CustomerID|CustomerName|OrderID|OrderAmount|
+----------+------------+-------+-----------+
|C101      |Rahul       |O001   |5000.0     |
|C102      |Priya       |O002   |3000.0     |
|C101      |Rahul       |O003   |2500.0     |
|C103      |Amit        |O004   |7000.0     |
|C104      |Sneha       |O005   |4000.0     |
|C102      |Priya       |O006   |1500.0     |
|C104      |Sneha       |O007   |2000.0     |
+----------+------------+-------+-----------+
```

c. Identify customers who have never placed an order.



```
).show(truncate=False)

# (c) Find customers who never placed an order
print("\n(c) CUSTOMERS WITHOUT ORDERS")

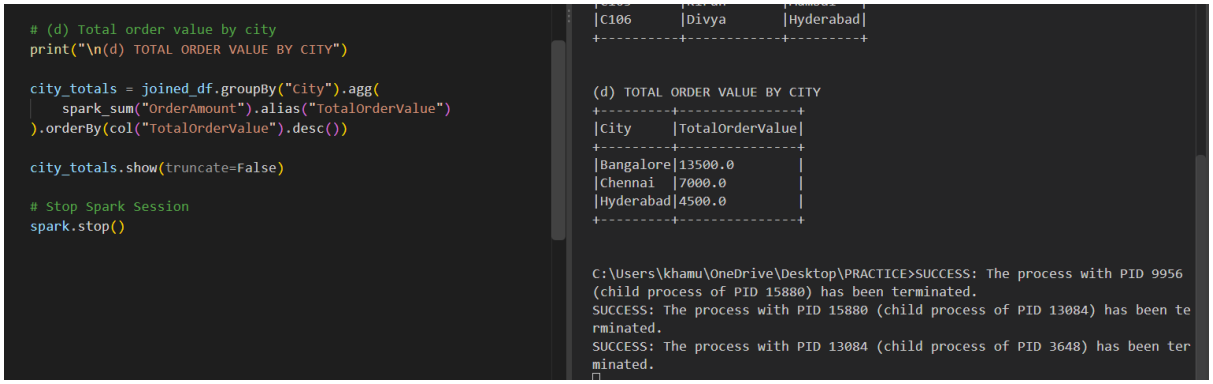
no_orders = customers.join(
    orders.select("CustomerID").distinct(),
    on="CustomerID",
    how="left_anti"
)

no_orders.show(truncate=False)
```

```
(c) CUSTOMERS WITHOUT ORDERS
[Stage 6:>                                                          (0 + 1)

+----------+------------+---------+
|CustomerID|CustomerName|City     |
+----------+------------+---------+
|C105      |Kiran       |Mumbai   |
|C106      |Divya       |Hyderabad|
+----------+------------+---------+
```

d. Calculate the total order value by city.



```
# (d) Total order value by city
print("\n(d) TOTAL ORDER VALUE BY CITY")

city_totals = joined_df.groupBy("City").agg(
    spark_sum("OrderAmount").alias("TotalOrderValue")
).orderBy(col("TotalOrderValue").desc())

city_totals.show(truncate=False)

# Stop Spark Session
spark.stop()
```

```
|C106      |Divya       |Hyderabad|
+----------+------------+---------+


(d) TOTAL ORDER VALUE BY CITY
+---------+---------------+
|City     |TotalOrderValue|
+---------+---------------+
|Bangalore|13500.0        |
|Chennai  |7000.0         |
|Hyderabad|4500.0         |
+---------+---------------+


C:\Users\khamu\OneDrive\Desktop\PRACTICE>SUCCESS: The process with PID 9956
(child process of PID 15880) has been terminated.
SUCCESS: The process with PID 15880 (child process of PID 13084) has been te
rminated.
SUCCESS: The process with PID 13084 (child process of PID 3648) has been ter
minated.
```

e. Explain when a broadcast join is useful and what conditions make it appropriate.

A broadcast join is a PySpark optimization in which a small DataFrame is sent to all worker nodes to join it with a larger DataFrame. It reduces data shuffling and improves performance.

Conditions: Use a broadcast join when one DataFrame is small enough to fit in each worker's memory and the other DataFrame is comparatively large.

f. Explain the difference between transformations and actions, giving one example of each.

Transformation : filtered = orders.filter(orders.OrderAmount > 3000)

Create a new DataFrame or RDD from existing data. [Ex: filter(), select(), join()]

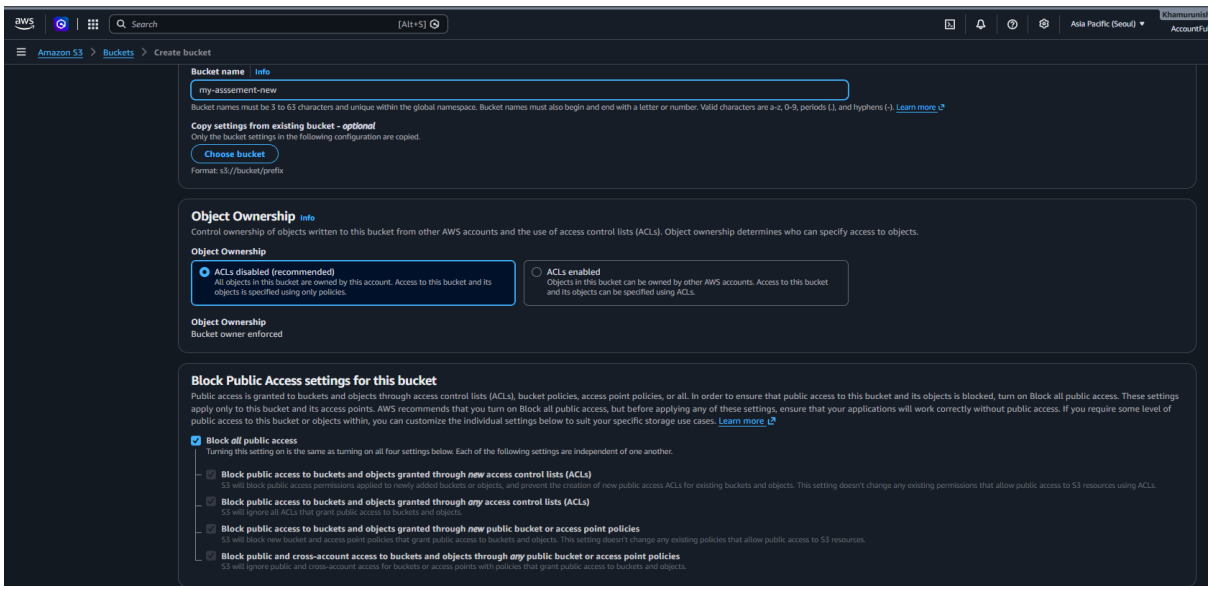
Action : filtered.show()

Execute computations to produce results or write data. [Ex: show(), count(), collect()]

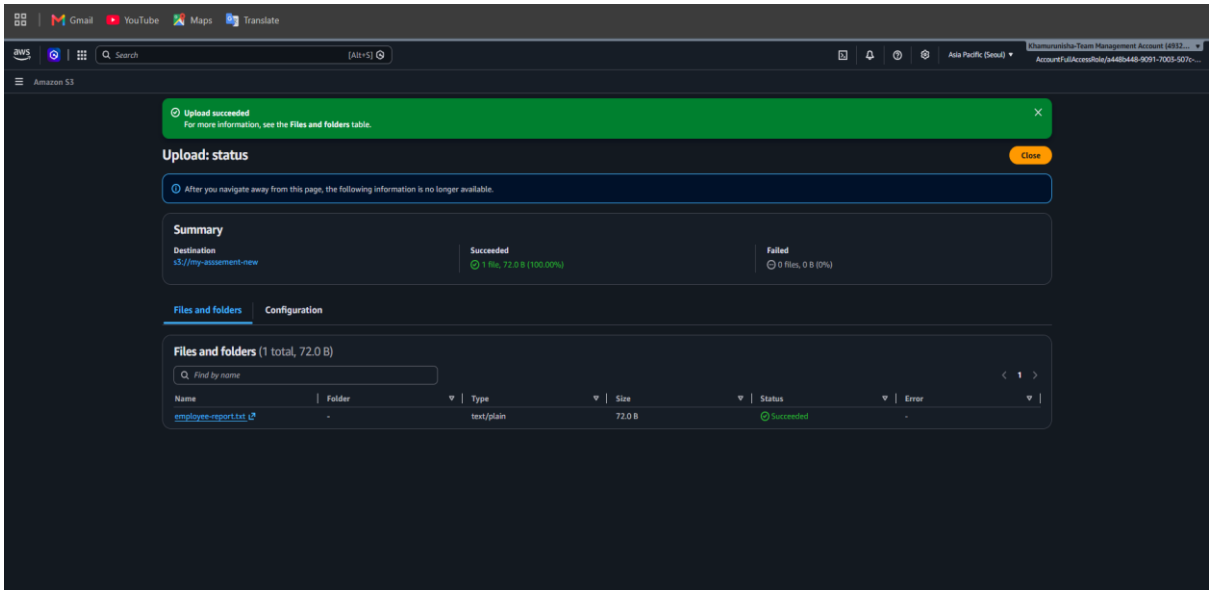
SECTION D: AWS CLOUD SERVICES

Question 10: S3 and IAM Security

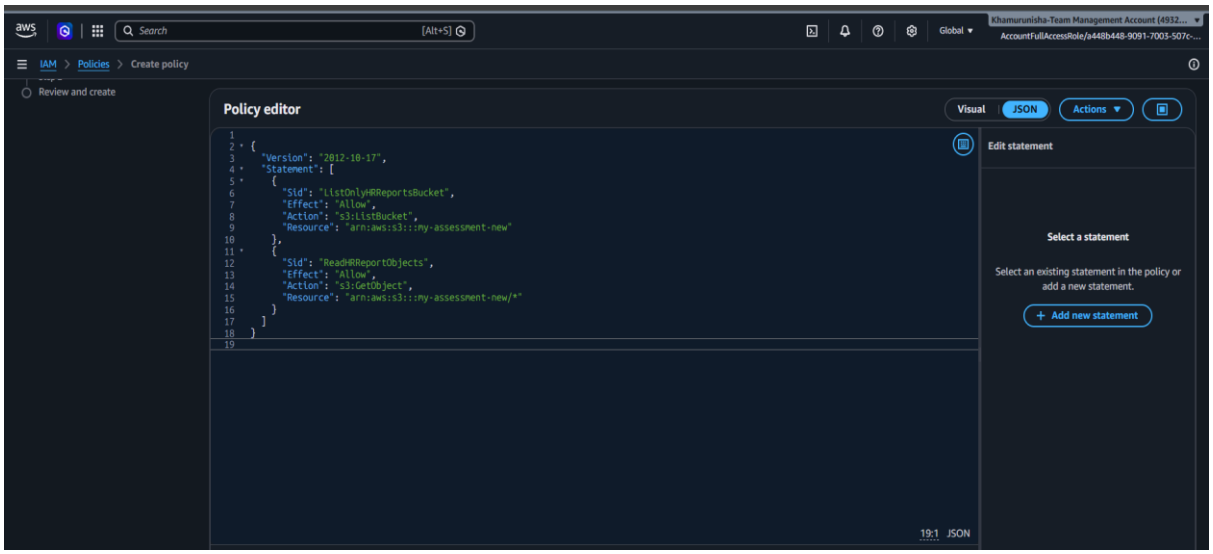
a. Create an S3 bucket with Block Public Access enabled and upload a sample file.



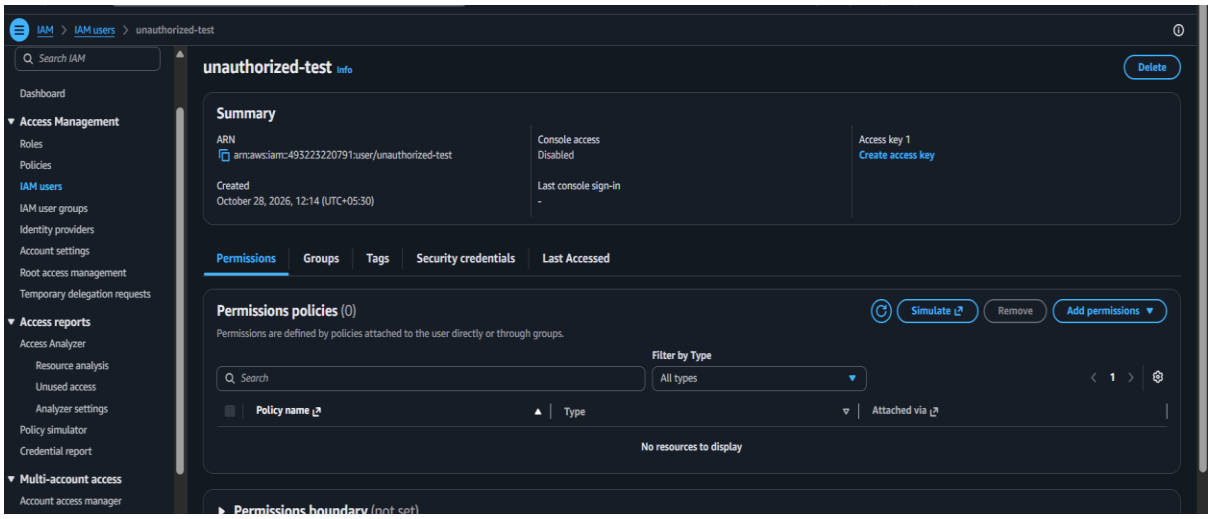
UPLOAD:



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.

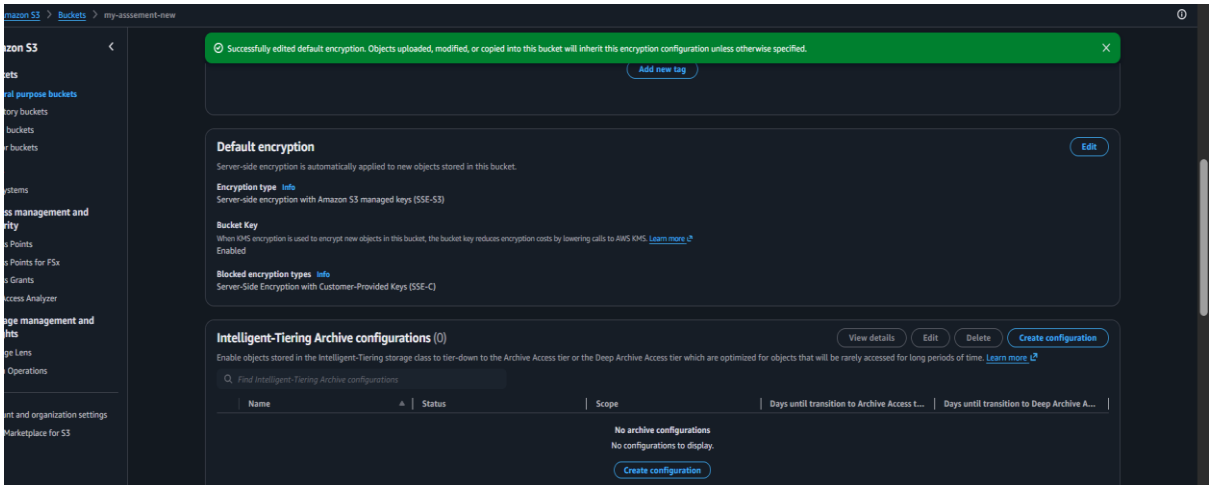


c. Explain how you would verify that an unauthorized user cannot read the file.



Sign in using an IAM test user or role that does not have permission to read the HR reports.

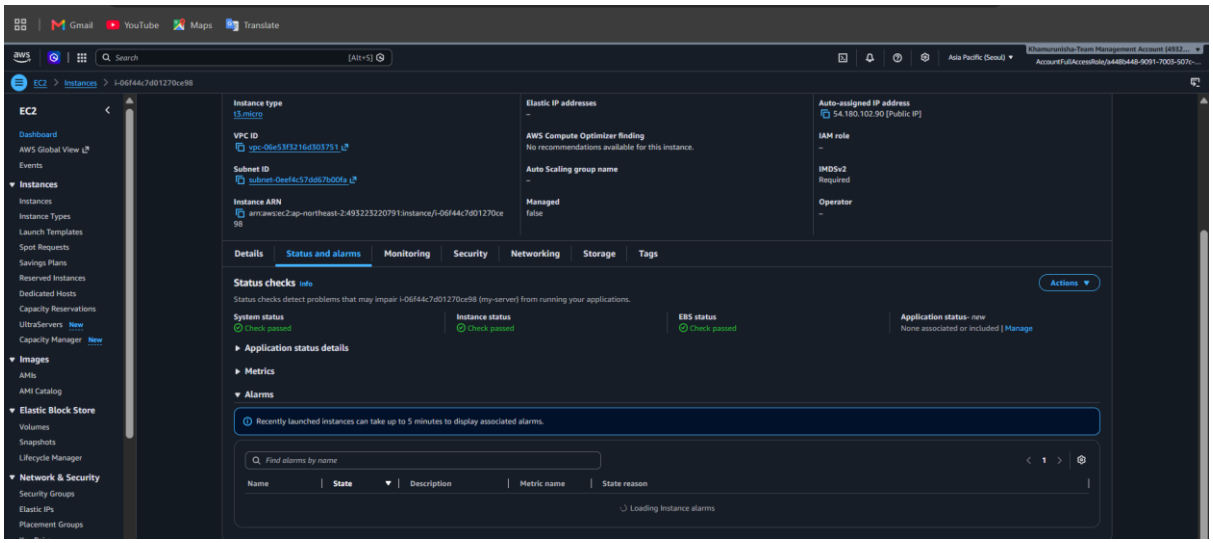
d. Explain how encryption at rest protects the stored data.



Encryption at rest protects data stored in Amazon S3 by converting it into unreadable encrypted form. S3 supports SSE-S3 and SSE-KMS to help protect confidential employee reports from unauthorized access.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.

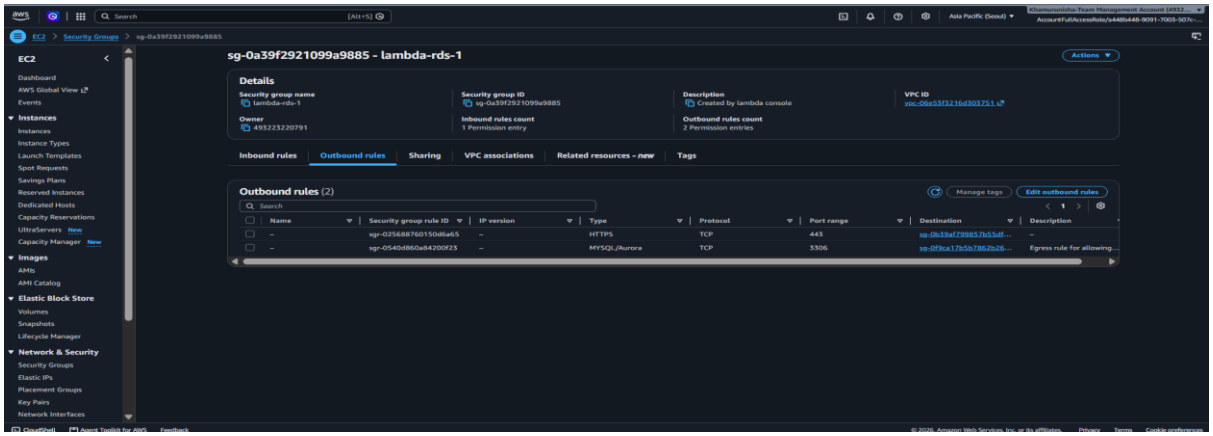


b. Explain how security groups and route tables can affect application connectivity.

A. **Security groups** act as virtual firewalls for EC2 instances. They control inbound [HTTP (port 80) or HTTPS (port 443) traffic is not allowed] and outbound traffic [external services].

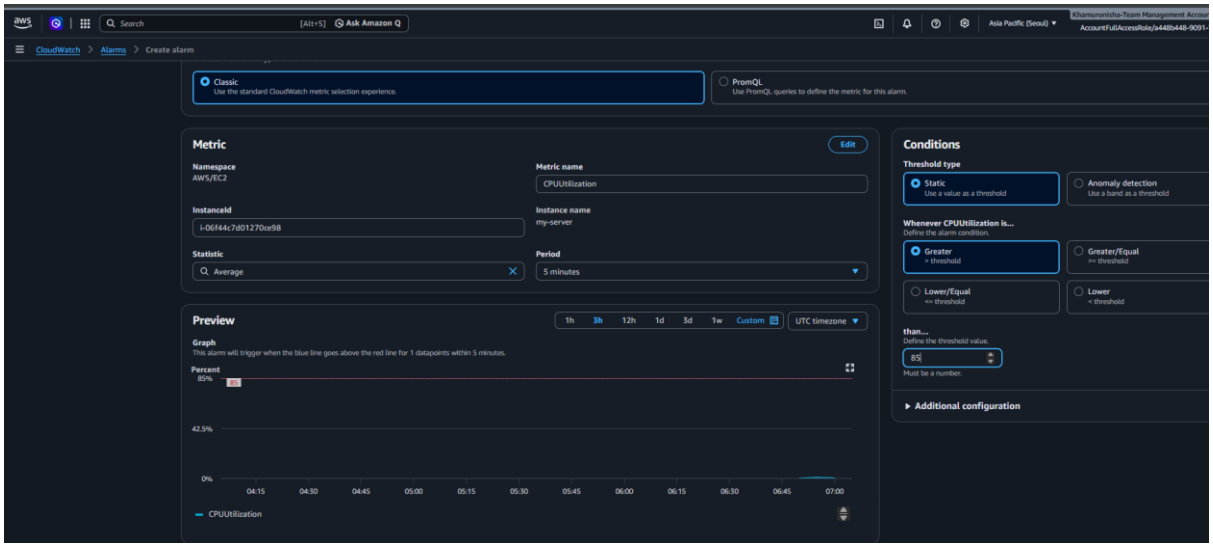
B. **Route tables** determine where network traffic is sent.

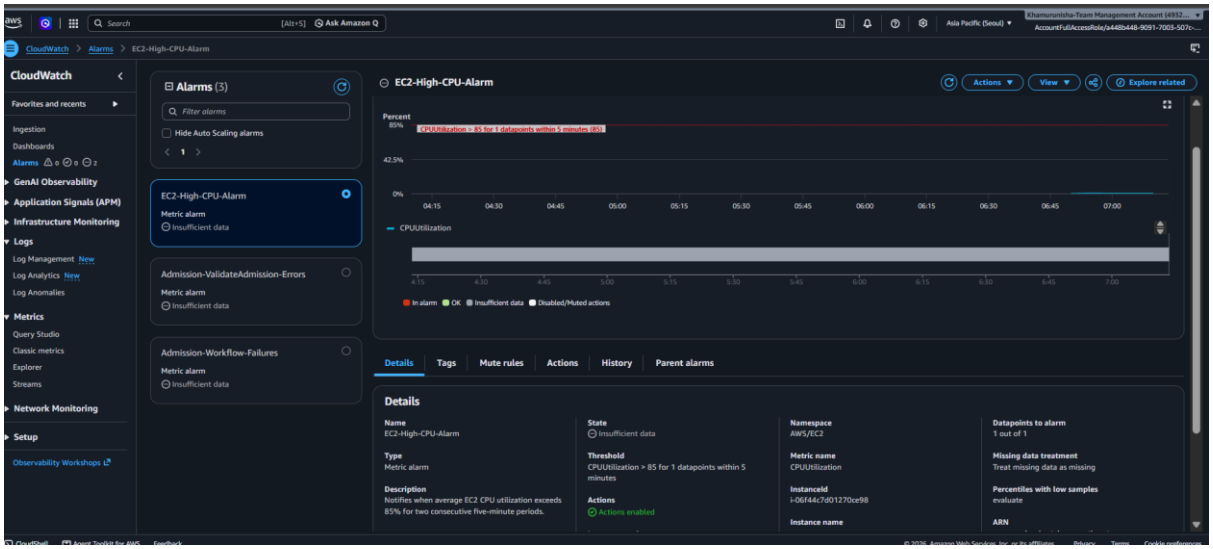
- A public subnet typically needs a route to an Internet Gateway (0.0.0.0/0) for direct internet connectivity.
- A private subnet generally needs suitable routing, such as through a NAT Gateway for outbound internet access.



c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

We'll create an alarm that triggers when the average CPU utilization exceeds 85% for two consecutive five-minute evaluation periods.

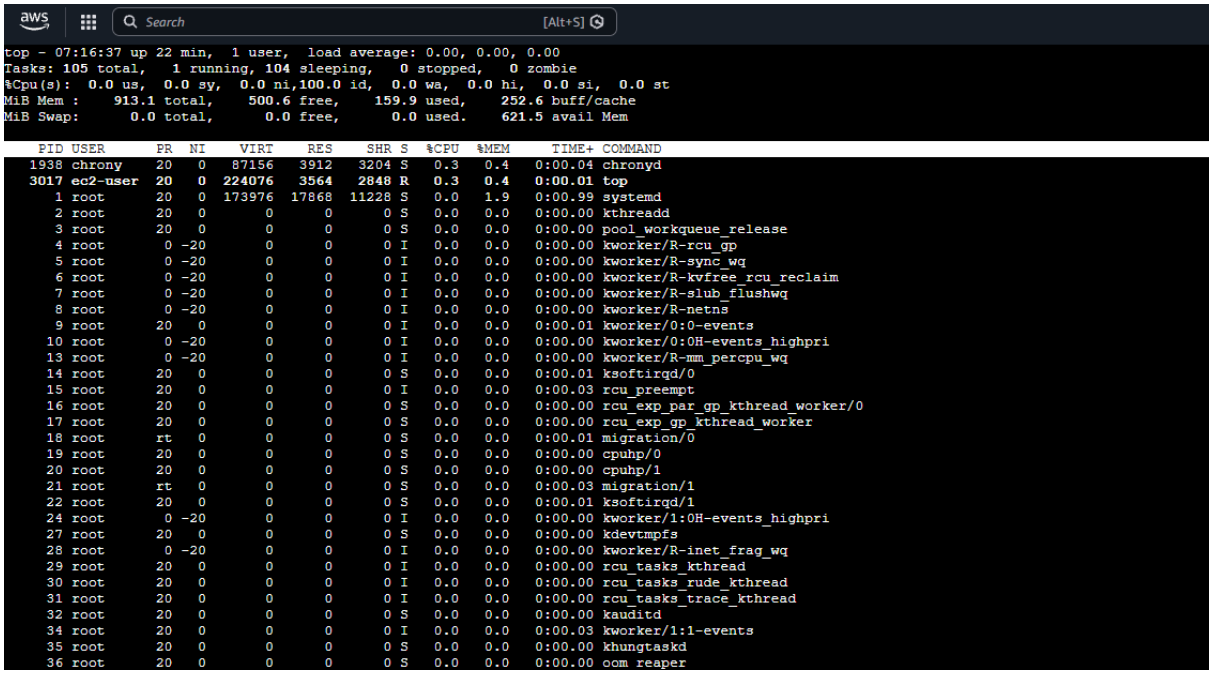




The alarm will notify your subscribed email when CPU utilization exceeds 85% for the configured evaluation period. You do not need to add a Lambda action for this assignment.

d. Identify one method for investigating the root cause of high CPU utilization.

Method: Using the Linux top command.



```
top - 07:16:37 up 22 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 105 total,   1 running, 104 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    913.1 total,    500.6 free,    159.9 used,    252.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    621.5 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1938 chrony    20   0   87156   3912   3204 S   0.3   0.4   0:00.04 chronyd
   3017 ec2-user  20   0  224076   3564   2848 R   0.3   0.4   0:00.01 top
      1 root      20   0  173976  17868  11228 S   0.0   1.9   0:00.99 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0.0   0.0   0:00.01 kworker/0:0-events
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.01 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.03 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.01 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.03 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.01 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     28 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
     29 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     30 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     31 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     32 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     34 root      20   0       0      0      0 I   0.0   0.0   0:00.03 kworker/1:1-events
     35 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     36 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
```

Question 12: Event-Driven Processing

a. Identify a suitable service for executing the processing code.

AWS Lambda is suitable because it executes code without requiring the company to manage servers. A Lambda function can read a CSV file, validate its contents, transform the data, and save the results to another S3 bucket.

b. Explain how an S3 upload can trigger processing.

- Configure an Amazon S3 event notification for the ObjectCreated event on the input bucket and set AWS Lambda as the destination. Whenever a CSV file is uploaded, S3 invokes the Lambda function, which processes the file.
- A suffix filter such as .csv can restrict triggering to CSV files.

c. Explain the IAM permissions needed to read the input file and write the results.

Create an IAM execution role for Lambda with a least-privilege policy.

- s3:GetObject on the input bucket's CSV objects.
- s3:PutObject on the output bucket's result objects.
- CloudWatch Logs permissions to create log streams and write logs.

d. Explain how errors and processing logs can be captured.

Use Amazon CloudWatch Logs to capture Lambda execution logs, including processing status, validation failures, and exception messages. Configure CloudWatch alarms for errors or repeated failures. For failed asynchronous invocations, configure a dead-letter queue or a Lambda destination to retain failure information for investigation.

e. Explain how to prevent duplicate processing or recursive triggering.

- Prevent duplicate processing: Make the Lambda function idempotent by recording processed object identifiers, such as the bucket, object key, and version ID, in a database or deduplication store. Check the record before processing again.

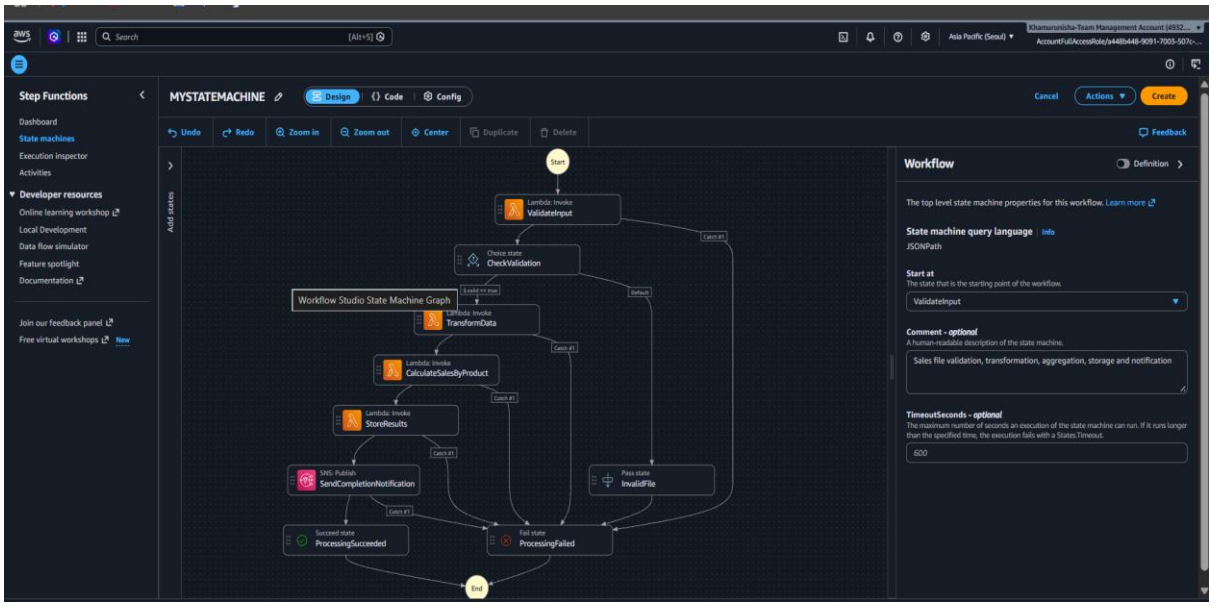
- Prevent recursive triggering: Store results in a separate output bucket, or configure the S3 event notification to trigger only for an input prefix such as incoming/ and write results under processed/.

SECTION E: AWS STEP FUNCTIONS

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification.

State	Purpose
ValidateInput — Task	Invokes Lambda to validate the sales file.
CheckValidation — Choice	Routes valid and invalid files.
TransformData — Task	Cleans and transforms sales records.
CalculateSales — Task	Calculates total sales grouped by product.
StoreResults — Task	Writes the results to Amazon S3.
SendNotification — Task	Publishes a completion message through Amazon SNS.
ProcessingFailed — Fail	Ends the workflow when validation or processing fails.
Success — Succeed	Marks successful completion.



b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

The CheckValidation Choice state checks the Boolean value $.validation.isValid.

- If isValid = true, execution proceeds to TransformData.
- If isValid = false, execution follows the default path to ProcessingFailed.

This prevents invalid files from entering the transformation and calculation stages.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

- **Retry**: Automatically retries a task when a temporary error occurs, such as throttling or a transient service failure. IntervalSeconds sets the initial delay, MaxAttempts limits retries, and BackoffRate increases the delay between attempts.
- **Catch**: Handles errors that remain after retries or other task failures. It captures error details in $.error and routes execution to ProcessingFailed.

For example, if a Lambda function temporarily fails , Step Functions can retry it. If it still fails, Catch routes the workflow to the failure state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

Step Functions uses Task states to invoke Lambda functions through the optimized Lambda integration, such as arn:aws:states:::lambda:invoke. It can also integrate directly with supported AWS services, such as Amazon SNS for notifications. An execution role grants permission to perform these operations**.**

e. Explain how execution history helps troubleshoot a failed workflow.

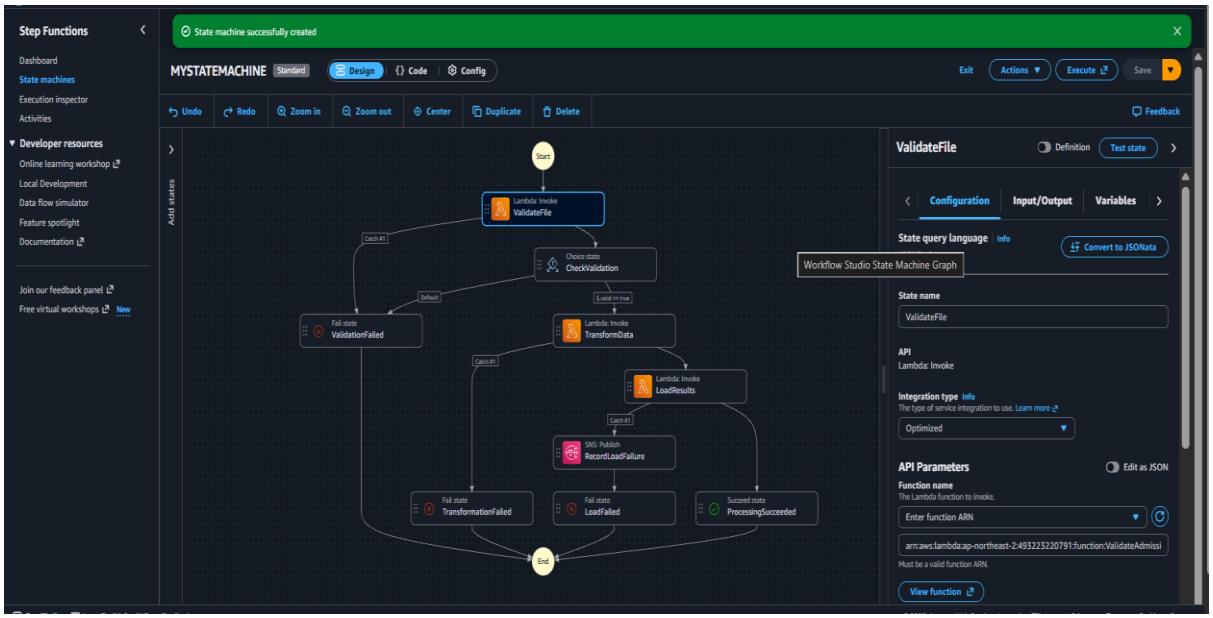
Step Functions execution history records state transitions, inputs, outputs, errors, and retry attempts. By opening the failed execution in the AWS console, a developer can identify the state that failed, inspect the error message, and

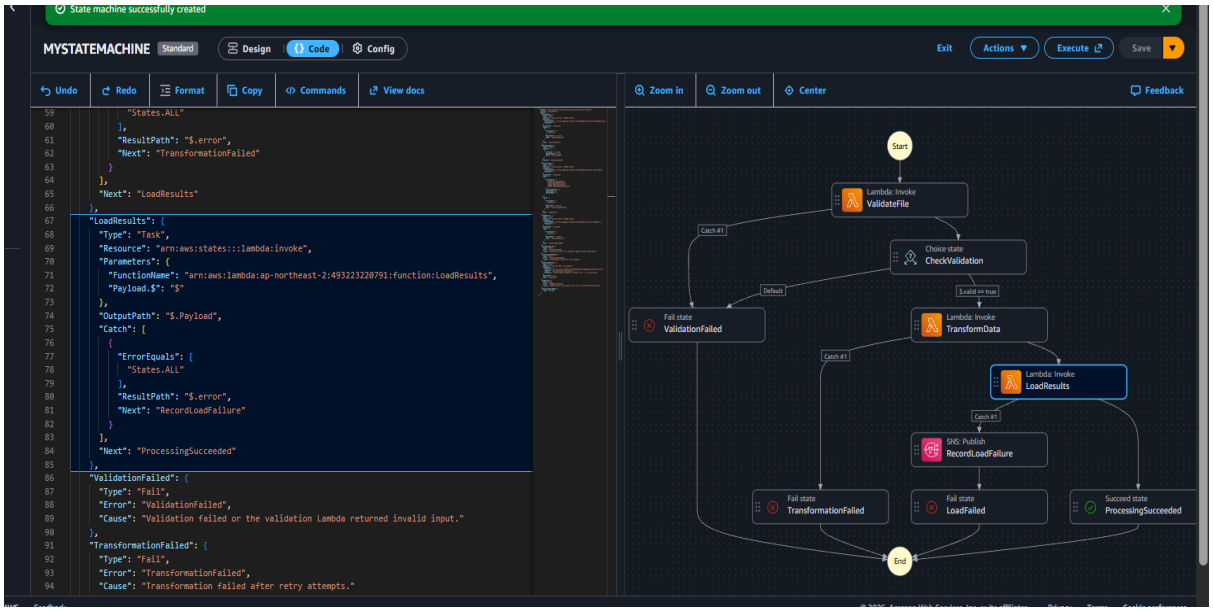
determine whether the issue occurred during validation, transformation, calculation, storage, or notification.

Question 14: Step Functions Implementation

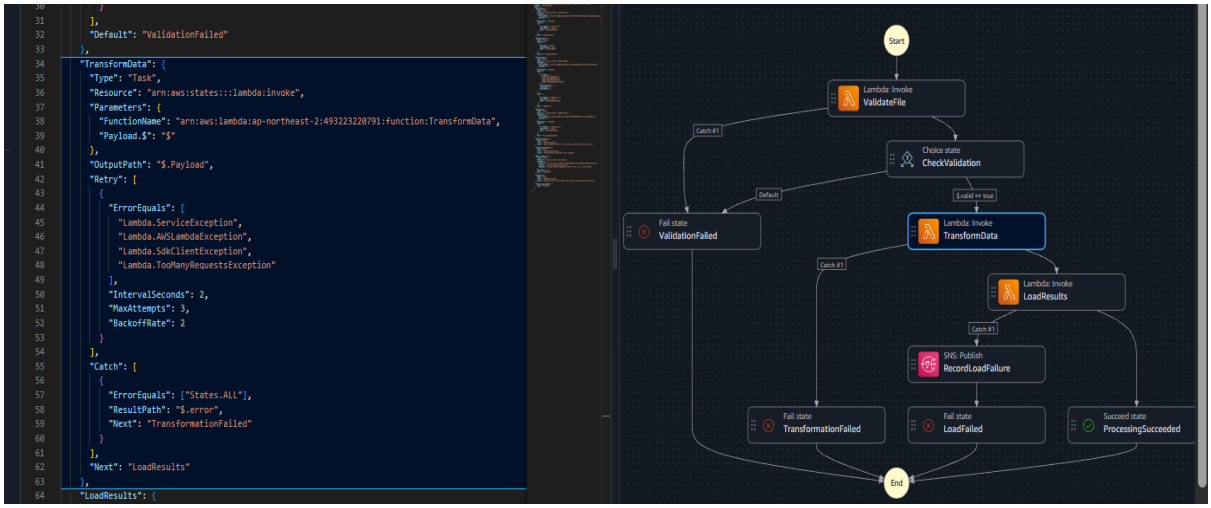
a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

The following AWS Step Functions definition contains the three sequential tasks: ValidateFile, TransformData, and LoadResults. It uses your AWS Region (ap-northeast-2) and account ID (493223220791), reuses your existing ValidateAdmission Lambda for the validation step, and uses placeholders for the other two Lambda functions.





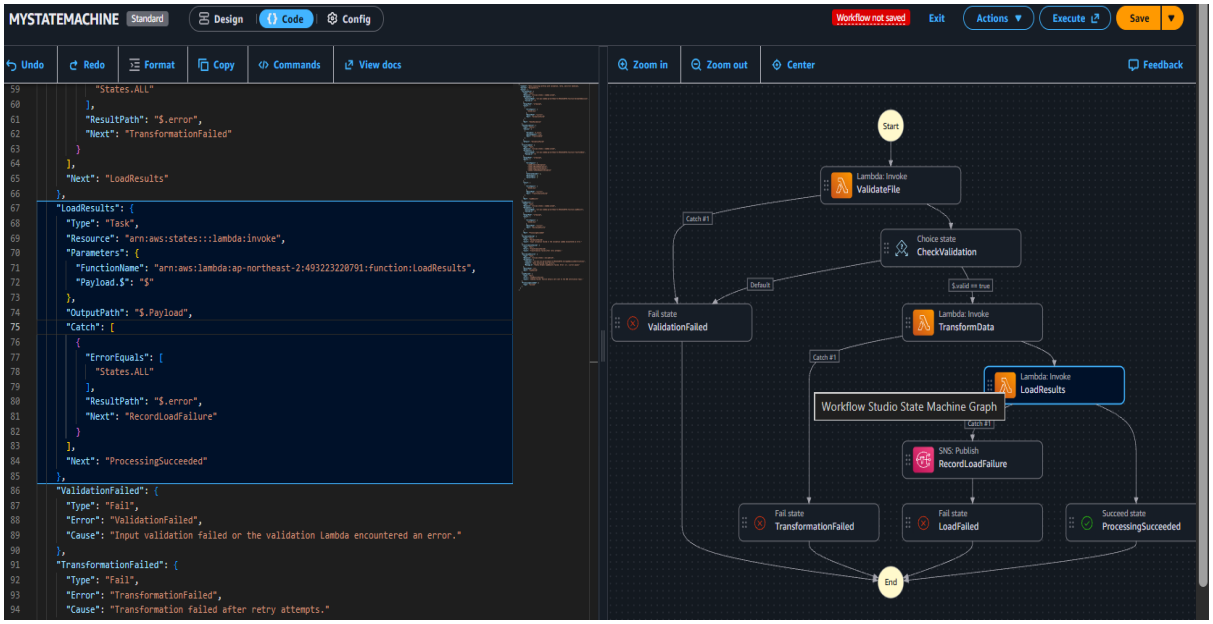
b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.



Setting	Value	Explanation
ErrorEquals	Lambda service and throttling errors	Specifies retryable errors
IntervalSeconds	2	Wait 2 seconds before the first retry
MaxAttempts	3	Retry up to 3 times
BackoffRate	2	Doubles the delay after each retry

This provides retry delays of approximately 2, 4, and 8 seconds if all three retries are needed.

c. Configure a Catch transition for the loading task that routes to an error-handling state.



- ErrorEquals: ["States.ALL"] catches errors from the loading task.
- ResultPath: "$.error" saves the error details in the workflow data.
- Next: "RecordLoadFailure" routes the workflow to the error-handling state.
- RecordLoadFailure publishes a failure notification through your existing SNS topic, CollegeAdmissionNotifications.
- LoadFailed then marks the execution as failed.

d. Explain how data from one task is passed to the next task.

- AWS Step Functions passes data between tasks through JSON input and output. Each task receives the previous task's output as its input.
- In our state machine, "Payload.$": "$" passes the current state input to the Lambda function, while "OutputPath": "$.Payload" extracts the Lambda function's response to pass to the next state.

{

"valid": true,

"file": "sales.csv"

}

If ValidateFile returns this data, TransformData receives it as input. The transformed output is then passed to LoadResults.

e. Explain how you would test successful and failed executions.

1. Successful execution: Run the Step Functions workflow with valid JSON input. Verify that all tasks execute successfully and the workflow reaches ProcessingSucceeded.

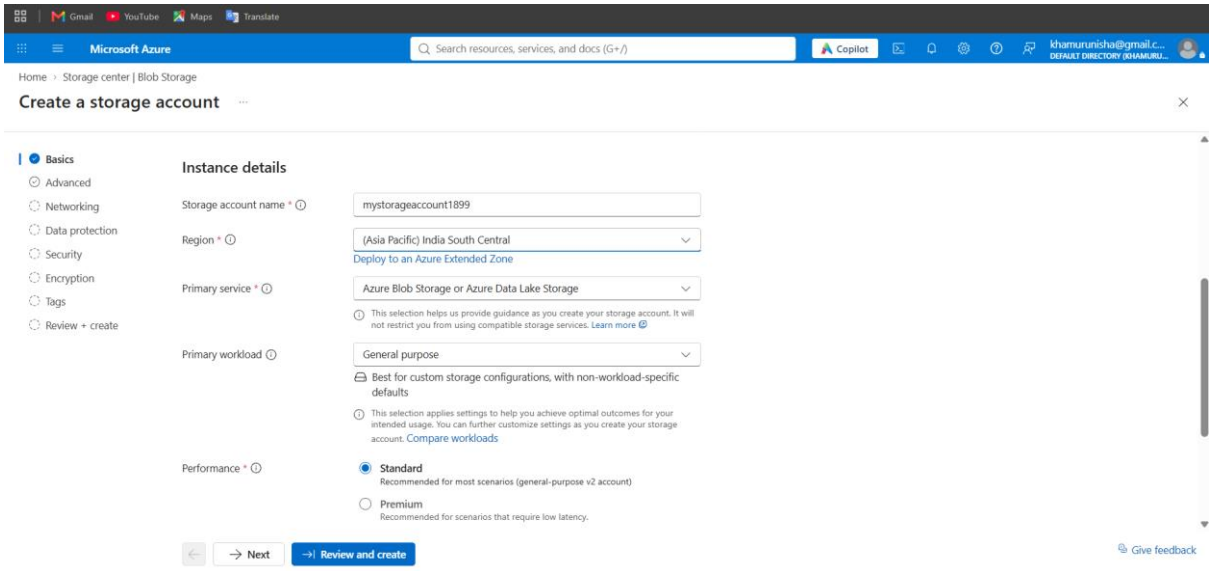
2. Failed execution: Run the workflow with invalid or incomplete input, such as a missing email. Verify that validation fails and the workflow reaches ValidationFailed. Check the execution graph and CloudWatch logs to identify errors and confirm error handling.

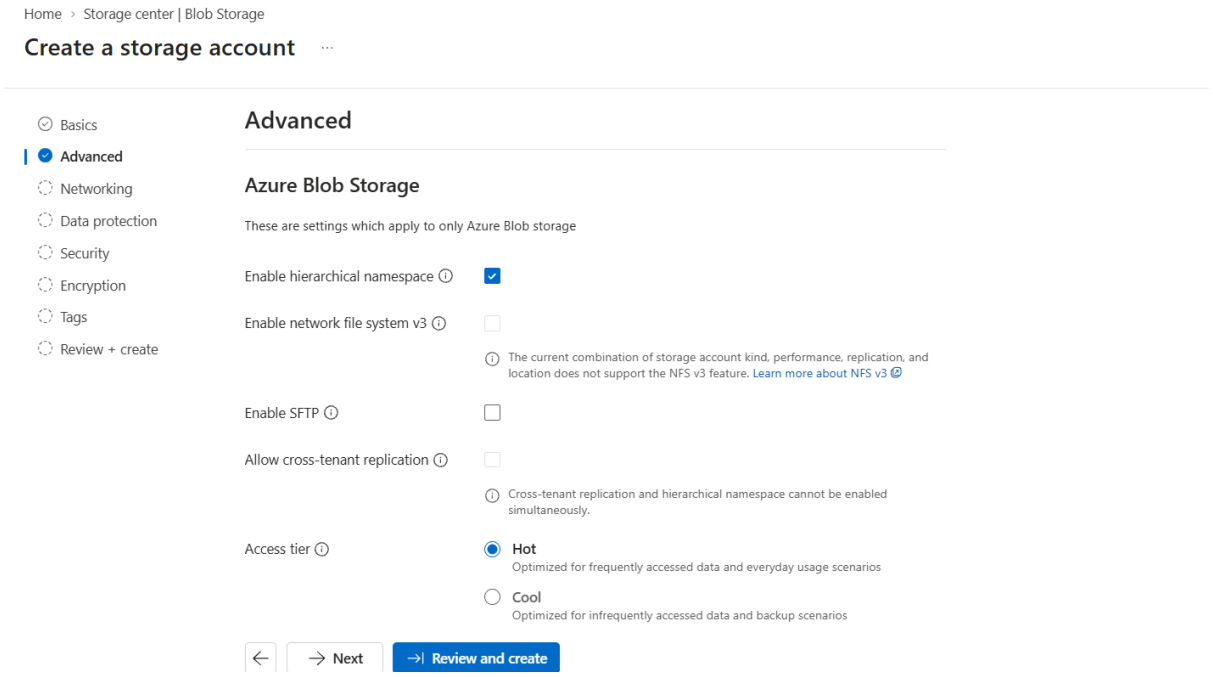


SECTION F: MICROSOFT AZURE

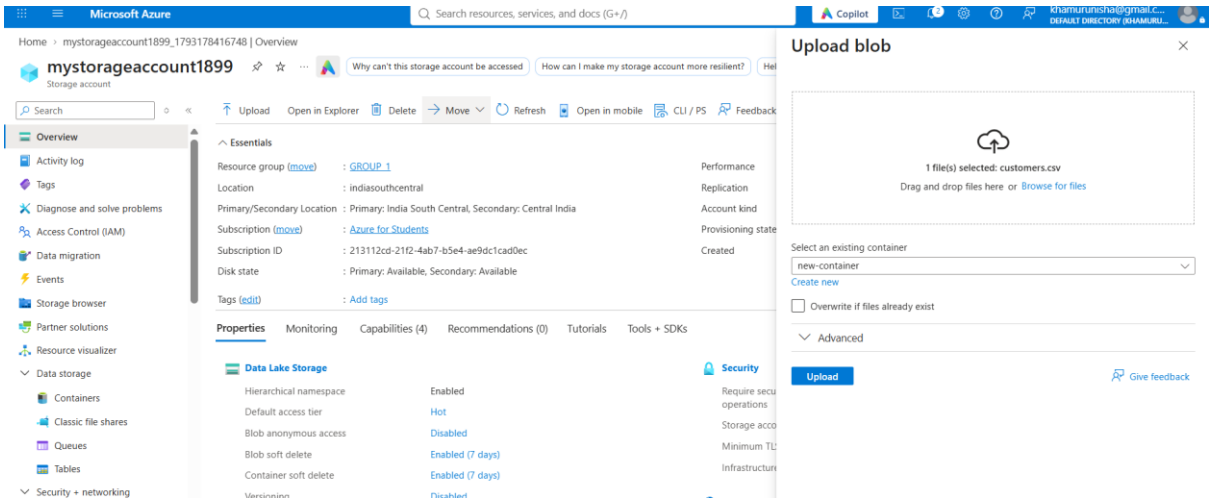
Question 15: Azure Data Lake Storage and Security

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.



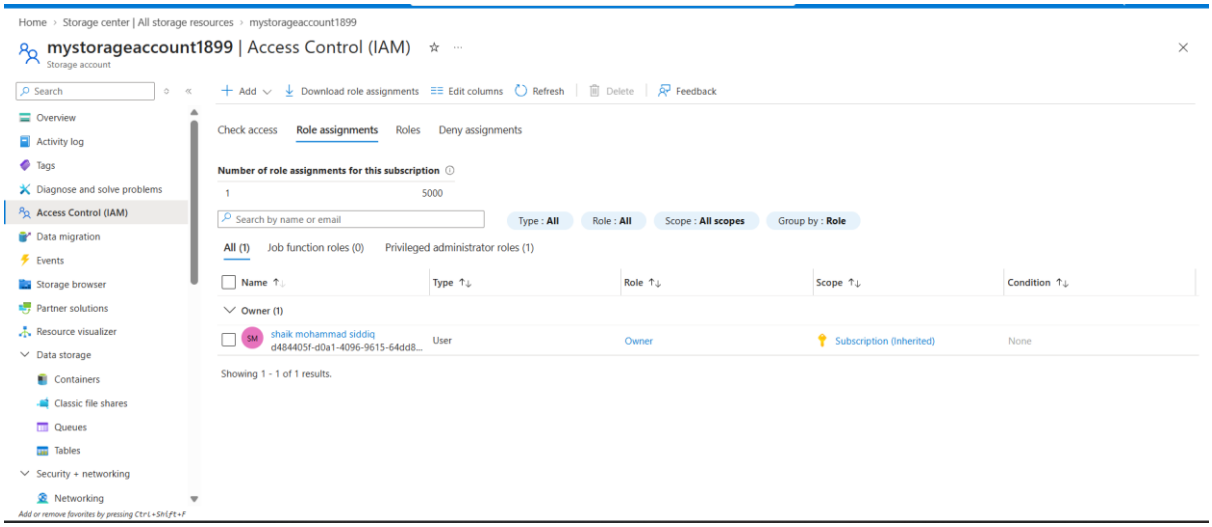


b. Create a filesystem or container and upload a sample dataset.



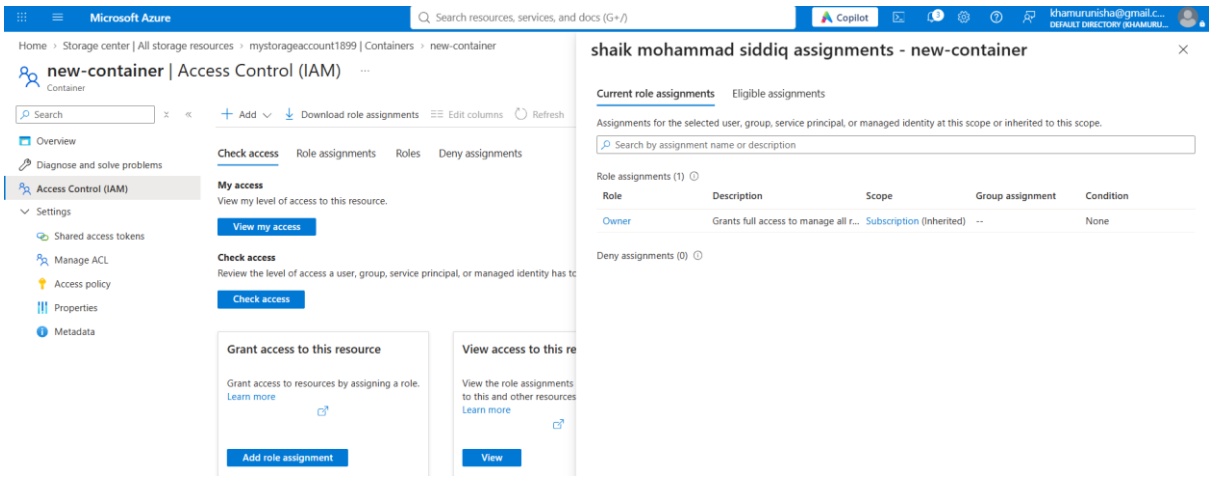
c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data.

- Microsoft Entra ID manages user identities, groups, and authentication.
- Azure RBAC assigns roles such as Storage Blob Data Reader for read-only access and Storage Blob Data Contributor for reading, writing, and deleting data.
- Assign permissions to authorized users or groups and follow the principle of least privilege.
- Use Azure Data Lake Storage access control lists (ACLs) when more granular file- and directory-level permissions are required.



d. Explain how you would verify that unauthorized users cannot access the dataset.

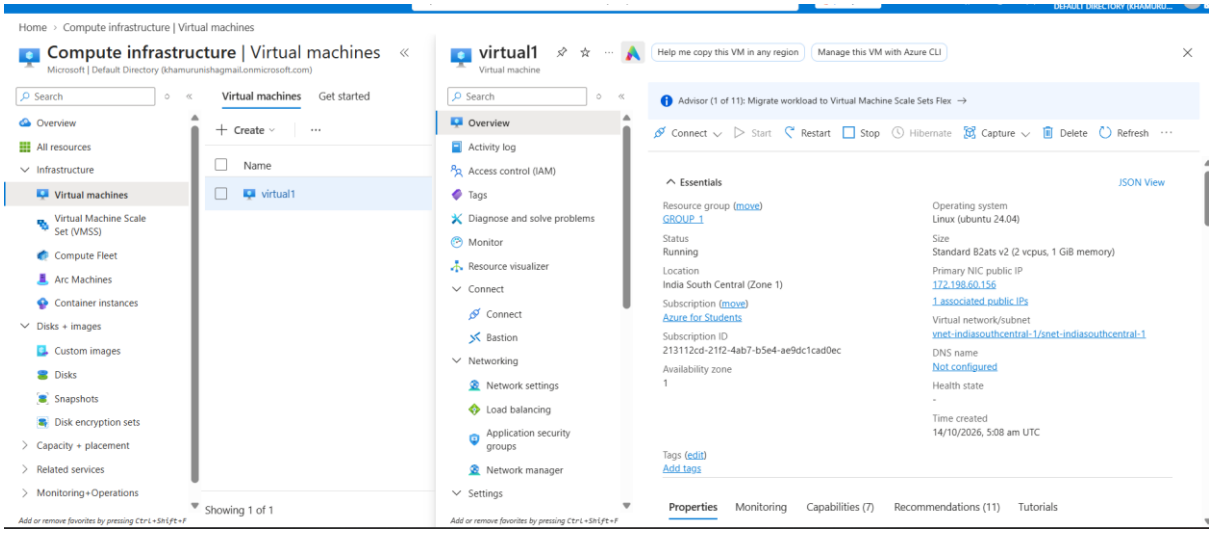
I would sign in as a user who does not have permission to access the Azure Data Lake Storage dataset and attempt to open or download the file. If access is denied with a 403 Forbidden error, it confirms that unauthorized access is restricted. I would also verify that an authorized user can access the dataset successfully.



Question 16: Azure VM Troubleshooting

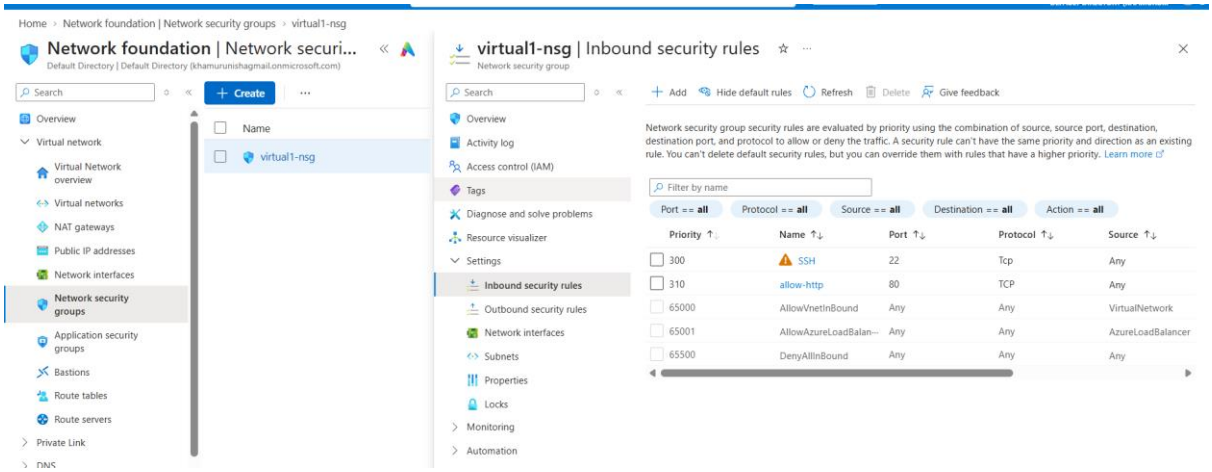
a. Verify the VM's provisioning and power states.

Open Azure Portal → Virtual Machines → Select the VM → Overview. Check the provisioning state is Succeeded and the power state is Running.



b. Explain how Network Security Group rules can prevent connectivity.

NSG rules may block connectivity if inbound traffic on required ports, such as SSH (22) or RDP (3389), is denied. Verify inbound rules, priorities, source IP addresses, and destination ports.

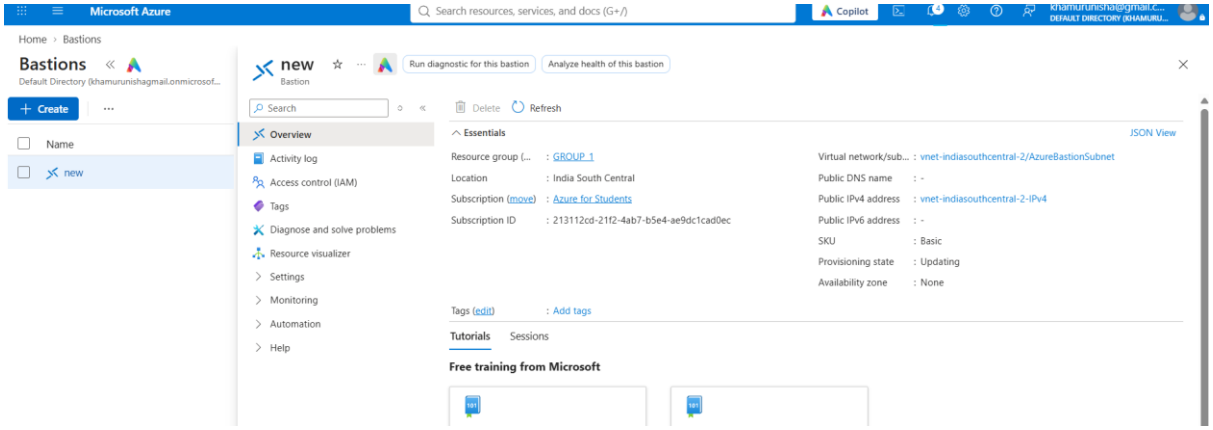


c. Explain how subnet and routing configuration affect connectivity.

Incorrect subnet configuration, route tables, or user-defined routes can prevent traffic from reaching the VM. Verify the subnet association, route table, next-hop settings, and network connectivity.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

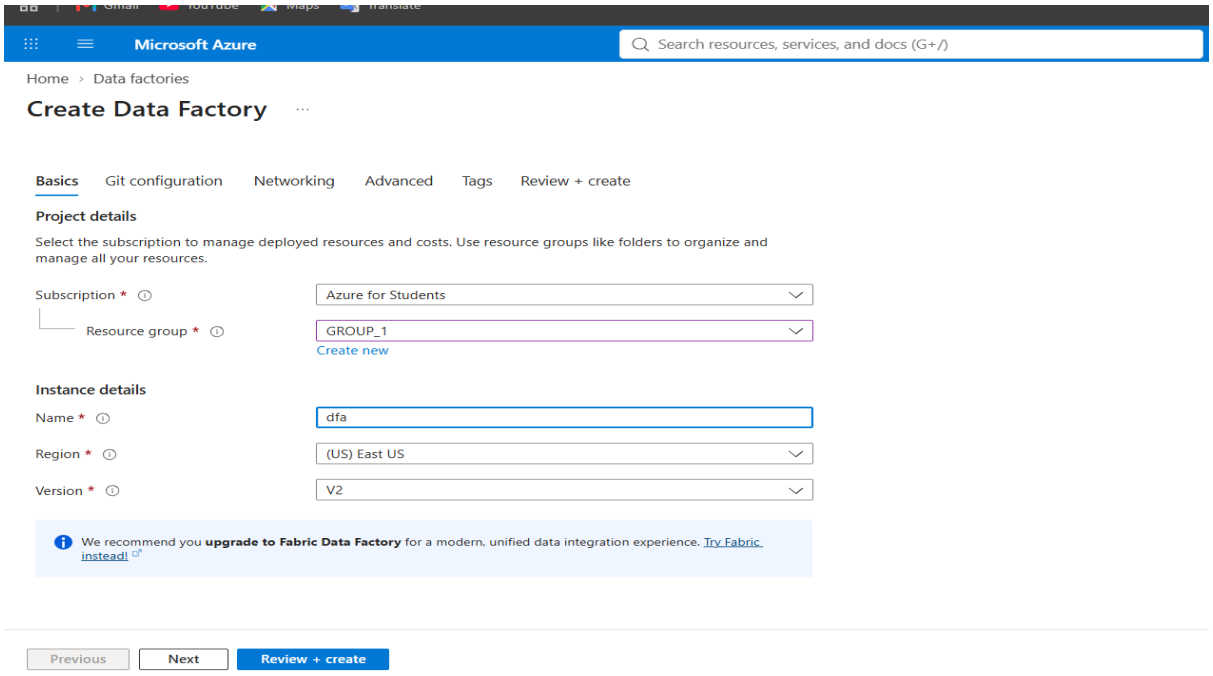
Use Azure Bastion to connect to the VM through the Azure Portal without exposing the VM's public IP or opening SSH/RDP ports to the public internet. Alternatively, use Azure Serial Console for supported troubleshooting scenarios. Review NSG flow logs or Network Watcher connection troubleshooting to identify blocked traffic, and grant access only to authorized administrators.



Question 17: Azure Data Integration

a. Select an appropriate service to orchestrate data movement.

Use Azure Data Factory (ADF) to create and schedule a data pipeline that moves customer transaction data from the source to Azure Data Lake Storage Gen2.



b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2.

Create a Copy Data activity in Azure Data Factory. Configure the CSV file as the source using a linked service and configure Azure Data Lake Storage Gen2 as the destination. Run the pipeline to copy the file.

c. Describe how to organize files by ingestion date.

Store files in date-based folders, such as:

```
customer-transactions/
  year=2026/
    month=10/
      day=28/
        transactions.csv
```

This makes data easier to manage, query, and analyze by ingestion date.

d. Explain how to handle malformed records and missing values.

Use Azure Data Factory Mapping Data Flows or Azure Databricks to validate and transform records. Send malformed records to a separate error folder, and handle missing values by filling defaults or rejecting records according to business rules.

e. Explain how to monitor pipeline execution and validate data completeness.

Use Azure Data Factory Monitor and Azure Monitor to check pipeline status, duration, and errors. Compare source and destination record counts and verify that all expected files were copied successfully.